

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

Специальность 6-05-0611-01 «Информационные системы
и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Распределенные информационные системы»

на тему: **Программное обеспечение для определения влажности торфа по
изображению**

Исполнитель: студент гр. ИТП-31
Проскудин А.О.

Руководитель: преподаватель
Комраков В.В.

Дата проверки: _____

Дата допуска к защите: _____

Дата защиты: _____

Оценка работы: _____

Подписи членов комиссии по защите
курсового проекта:

Гомель 2026

Установа адукацыі «Гомельскі дзяржаўны тэхнічны ўніверсітэт імя П.В. Сухого»

Факультэт аўтаматызаваных і інфармацыйных сістэм

Кафедра "Информационные технологии"

РЭЦЭНЗІЯ
на курсавы праект (работу)

па дысцыпліне «Разработка приложений баз данных»,

выканана студэнтам _____

групы _____

I. Пералік заўваг па тэксту курсавога праекта (работы)

II. Агульная характарыстыка работы

СОДЕРЖАНИЕ

Введение	5
1 Обзор технологий и обоснование архитектуры приложения	6
1.1 Предметная область и выбор регрессионного подхода к анализу влажности.....	6
1.2 Выбор архитектуры нейронной сети	8
1.3 Выбор программной среды и языка программирования	11
2 Проектирование и реализация интеллектуальной системы	14
2.1 Выбор оборудования и условия получения датасета	14
2.2 Формирование датасета и алгоритмы предварительной обработки данных	17
2.3 Проектирование регрессионной модели и процесса обучения.....	21
2.4 Разработка графического интерфейса и модуля предсказания	23
3 Верификация и тестирование программного комплекса	26
3.1 Анализ процесса обучения по графикам функции потерь.....	26
3.2 Оценка качества предсказаний и тестирование интерфейса	28
Заключение.....	31
Список использованных источников	32
Приложение А Листинг программы	33
Приложение Б Руководство пользователя	44
Приложение В Руководство программиста	46
Приложение Г Внешний вид графического интерфейса программы	48

ВВЕДЕНИЕ

Торф является ценным природным ресурсом, который широко применяется как в энергетической промышленности в качестве эффективного топлива, так и в сельском хозяйстве для улучшения структуры почв и производства органических удобрений. Ключевым параметром, определяющим качество, теплотворную способность и пригодность данного сырья для дальнейшей переработки, выступает его влажность [1]. Традиционные лабораторные методы измерения содержания воды требуют значительных временных затрат, использования специализированного сушильного оборудования и физического изъятия проб, что делает невозможным оперативный контроль на конвейерной ленте или в полевых условиях. Внедрение алгоритмов компьютерного зрения позволяет бесконтактно и мгновенно оценивать состояние материала по его визуальным характеристикам, существенно снижая издержки и повышая общую эффективность производственных процессов из-за чего эта задача актуальна [2][3].

Целью данной курсовой работы является создание программного продукта с графическим интерфейсом, который будет решать задачу автоматического предсказания непрерывного значения влажности по цифровой фотографии. Для достижения намеченной цели требуется выполнить ряд исследовательских и инженерных шагов, начиная с проведения подробного обзора предметной области и анализа существующих методов визуальной оценки природных материалов. Вслед за этим необходимо осуществить проектирование алгоритмов подготовки данных, включающее обработку, масштабирование и аугментацию изображений для корректного обучения алгоритма. Центральным этапом работы выступает реализация регрессионной нейросетевой модели, способной выявлять скрытые нелинейные зависимости между цвето-текстурными признаками сырья и уровнем его увлажненности. После настройки вычислительного ядра требуется выполнить создание пользовательского интерфейса для удобного взаимодействия с обученной сетью, а затем провести итоговое тестирование системы с целью оценки ее точности и надежности на независимом наборе данных.

Успешная практическая реализация задуманного проекта опирается на применение современных программных технологий и инструментов машинного обучения. В качестве основного языка разработки используется *Python*, обладающий обширной экосистемой библиотек для работы с медиафайлами и массивами данных. Проектирование архитектуры и процесс обучения искусственной нейронной сети осуществляются через фреймворк *PyTorch*, предоставляющий высокую производительность при вычислениях на тензорах. Визуальная часть программы реализуется через доступные средства разработки десктопных приложений, что позволяет гармонично объединить сложную математическую логику с интуитивно понятным визуальным представлением.

1 ОБЗОР ТЕХНОЛОГИЙ И ОБОСНОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

1.1 Предметная область и выбор регрессионного подхода к анализу влажности

Физико-химическая структура торфа характеризуется высокой пористостью, что обуславливает его выраженную способность к поглощению и удержанию влаги. При увеличении содержания воды в массе материала происходят существенные изменения его оптических свойств, которые могут быть зарегистрированы как визуально, так и аппаратно. По мере заполнения внутренних капилляров и микропор молекулами воды изменяется относительный показатель преломления на границе раздела сред, что закономерно приводит к снижению общей отражающей способности поверхности сырья. Внешне данный физический процесс проявляется в виде заметного потемнения структуры торфа, сдвига его цветового тона в сторону более насыщенных, глубоких оттенков и существенного изменения контрастности микроструктуры. Влага постепенно обволакивает твердые частицы, сглаживая мелкие шероховатости и формируя на их поверхности микроскопические пленки. Эти пленки локально изменяют характер рассеяния света с диффузного на зеркальный, скрадывая первоначальный рельеф сухого сырья и служа важным оптическим маркером для автоматизированного визуального анализа. Наглядное различие между сухим (слева) и переувлажненным (справа) материалом представлено на рисунке 1.1.



Рисунок 1.1 – Визуальное сравнение образцов торфа с различной влажностью

Традиционные подходы к компьютерному зрению зачастую опираются на методы классификации, предполагающие отнесение анализируемых объектов к заранее заданным дискретным категориям. Однако в контексте

инструментальной оценки состояния торфа применение жестких категориальных рамок является методологически неоптимальным решением. Разделение исследуемого сырья на условные классы, например, на сухое, умеренно влажное и переувлажненное, неизбежно приводит к значительной потере точности, поскольку такой алгоритм полностью игнорирует тонкие градации физического состояния материала внутри этих искусственно введенных интервалов и на их границах. Влажность по своей фундаментальной физической природе представляет собой строго непрерывную величину, способную плавно изменяться и принимать любые рациональные значения в пределах допустимого диапазона. Следовательно, искусственная дискретизация данного параметра существенно снижает общую разрешающую способность диагностической системы и ограничивает возможности детализированного контроля производственного цикла.

Ввиду непрерывного характера исследуемого физического параметра, наиболее адекватным и обоснованным математическим аппаратом для решения поставленной проблемы выступает нейросетевая регрессия. Этот подход позволяет моделировать сложные нелинейные зависимости между многомерным пространством пиксельных признаков цифрового изображения и искомым количественным показателем содержания воды в образце. Применение регрессионной архитектуры искусственной нейронной сети позволяет алгоритму эффективно улавливать малейшие промежуточные состояния материала, обусловленные плавным изменением его отражательной способности и оптического контраста при впитывании влаги. В отличие от систем классификации, вычислительная система на базе регрессии приобретает способность генерировать в качестве выходного сигнала не абстрактную метку принадлежности к широкому классу, а предсказывать точное числовое значение. Такая парадигма машинного обучения математически точно отражает физическую реальность, обеспечивает максимальную информативность и достоверность результатов анализа, что делает разрабатываемую программную систему значительно более гибкой и применимой на практике при решении реальных задач промышленного и сельскохозяйственного мониторинга.

В рамках данного исследования целевой переменной выступает не классическая процентная влажность, а аппаратный индекс влажности (условные единицы АЦП), полученный напрямую с аппаратного емкостного датчика при формировании набора данных. Значения в датасете варьируются в диапазоне от 1720 до 2650 условных единиц, которые соответствуют 12-битному диапазону аналого-цифрового преобразователя микроконтроллера. Использование «сырых» показаний датчика в качестве целевой переменной для регрессионной нейросети является оправданным подходом, так как диэлектрическая проницаемость торфа строго и нелинейно коррелирует с объемом воды в его порах. Таким образом,

нейросеть обучается предсказывать непосредственно физический отклик измерительного прибора на основе визуальных характеристик материала.

1.2 Выбор архитектуры нейронной сети

Решение задачи автоматического извлечения информативных визуальных признаков из цифровых фотографий торфа для последующего предсказания непрерывного значения влажности требует тщательного и обоснованного выбора архитектуры сверточной нейронной сети. Анализ изображений неоднородных природных материалов с выраженной микротекстурой представляет собой нетривиальную вычислительную проблему, поскольку нейросетевая модель должна обладать способностью формировать многоуровневое иерархическое представление паттернов, улавливая как локальные высокочастотные изменения цветового тона, так и глобальную структуру поверхности увлажненного сырья. В процессе проектирования вычислительного ядра разрабатываемой системы был проведен подробный сравнительный анализ нескольких фундаментальных подходов к построению глубоких сверточных сетей, охватывающий как тяжеловесные классические архитектуры, так и современные легковесные решения, оптимизированные для мобильных платформ.

Рассмотрение тяжеловесных архитектур, ярким представителем которых является сверточная сеть *VGG16*, продемонстрировало их избыточность для решения поставленной задачи. Данная модель характеризуется последовательным применением однородных сверточных фильтров малого размера и обладает колоссальным объемом параметров, превышающим сто тридцать миллионов весовых коэффициентов. Несмотря на высокую способность к извлечению сложных пространственных признаков, применение *VGG16* в задачах анализа статической текстуры торфа нецелесообразно. Столь масштабное пространство параметров требует огромных вычислительных мощностей, значительных объемов видеопамати и критически увеличивает время логического вывода. Более того, при обучении на специализированных, ограниченных по объему наборах данных, столь емкая архитектура неизбежно сталкивается с проблемой сильного переобучения, когда сеть начинает запоминать информационный шум конкретных обучающих примеров вместо обобщения фундаментальных физических закономерностей, связывающих цвет и микрорельеф с реальной влажностью.

В качестве альтернативы тяжеловесным моделям были детально исследованы легковесные архитектуры, такие как *MobileNet* и семейство *EfficientNet*. Архитектура *MobileNet* базируется на математической концепции пространственно-разделяемых сверток, что позволяет радикально сократить количество умножений и вес модели, делая ее идеальной для внедрения на маломощных встраиваемых устройствах. Однако пространственное разделение каналов зачастую

приводит к потере тонких, высокочастотных деталей изображения, которые в случае анализа торфа несут критически важную информацию о наличии микроскопических пленок воды и специфическом слипании частиц. Модели семейства *EfficientNet*, использующие метод составного масштабирования базовой сети, демонстрируют выдающиеся результаты в задачах дискретной классификации, однако в задачах непрерывной нелинейной регрессии на текстурных данных они нередко требуют сложнейшей гиперпараметрической настройки и могут проявлять вычислительную нестабильность при адаптации последнего полносвязного слоя под вывод единственного скалярного значения. Сводные метрики рассмотренных глубоких нейросетевых моделей, наглядно отражающие их вычислительную емкость и применимость к разрабатываемой системе, представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ архитектур нейронных сетей

Архитектура	Количество параметров, млн	Вычислительная сложность, <i>GFLOPs</i>	Риск переобучения на малых данных	Пригодность для регрессии текстур
<i>VGG16</i>	138,4	15,50	Высокий	Низкая
<i>MobileNetV2</i>	3,4	0,30	Низкий	Средняя
<i>EfficientNet-B0</i>	5,3	0,39	Средний	Средняя
<i>ResNet34</i>	21,8	3,60	Умеренный	Высокая

Опираясь на количественные и качественные данные, представленные в таблице 1.1, в качестве вычислительного ядра системы была выбрана архитектура *ResNet34*, представляющая собой наиболее рациональный и математически обоснованный компромисс между репрезентативной способностью и вычислительной ресурсоемкостью. Главным технологическим прорывом семейства сетей *ResNet* является концепция остаточных связей, которая изящно решает фундаментальную проблему затухания и взрыва градиентов, неизбежно возникающую при обучении глубоких нейронных сетей методами обратного распространения ошибки. В классических последовательных архитектурах каждый новый слой пытается выучить полное преобразование входных данных, что при увеличении глубины приводит к экспоненциальному затуханию сигнала ошибки, делая настройку начальных слоев практически невозможной. Введение остаточных связей меняет эту парадигму: входной тензор передается не только через нелинейные сверточные фильтры, но и дублируется в обход них через тождественное преобразование, после чего суммируется с выходом блока. Таким образом, слои

обучаются предсказывать не само итоговое представление признаков, а лишь разницу, то есть остаток между входом и выходом. Это обеспечивает беспрепятственный поток градиентов по всей глубине сети, позволяя алгоритму уверенно сходиться к глобальному оптимуму даже при анализе сложнейших нелинейных зависимостей влажности.

Детальная структура вычислительного конвейера и внутренняя организация базовых модулей сети наглядно представлены на рисунке 1.2.

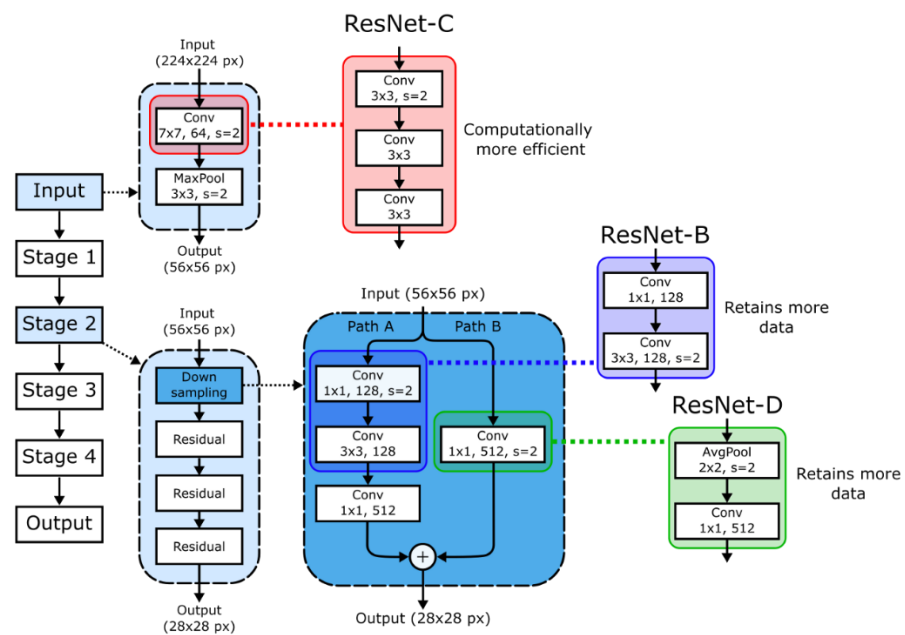


Рисунок 1.2 – Обобщенная схема и анатомия вычислительных блоков архитектуры *ResNet*

С математической точки зрения процесс прохождения данных через архитектуру представляет собой строго упорядоченную последовательность трансформаций, разделенную на несколько ключевых макроэтапов (от *Stage 1* до *Stage 4*). На первичном этапе входной тензор цифрового изображения обрабатывается сверточным фильтром размерности 7x7 и операцией максимальной субдискретизации (*MaxPool*), что позволяет алгоритму мгновенно подавить высокочастотный оптический шум и извлечь базовые градиенты цвета. Дальнейшая обработка переходит в каскад остаточных блоков. Как показано на детализированной схеме этапа понижения размерности (*Downsampling*), информационный поток внутри остаточного модуля логически разделяется на два параллельных маршрута. Основной вычислительный маршрут (*Path A*) осуществляет извлечение сложных пространственных признаков посредством серии нелинейных сверточных операций, тогда как альтернативный маршрут (*Path B*) формирует ту самую остаточную связь.

Важной архитектурной особенностью, отраженной на схеме, является механизм обработки тензоров на границах макроэтапов, где происходит сжатие пространственного разрешения. В таких узлах маршрут остаточной связи (*Path B*) не может быть прямым тождественным отображением из-за несовпадения размерностей матриц. Для решения этой проблемы применяются специализированные модификации блока (обозначенные как *ResNet-B* и *ResNet-D*), включающие дополнительные операции усредняющего пулинга или единичных сверток с шагом (*Conv 1x1, s = 2*). Применение таких адаптивных связей позволяет максимально сохранить исходные данные при их передаче на более глубокие слои. На завершающей стадии вычислений внутри каждого блока многомерные тензоры из обоих маршрутов объединяются с помощью поэлементной операции сложения, после чего результирующий вектор передается на следующий уровень абстракции.

На завершающем этапе обработки (*Output*) сформированный многомерный тензор глубинных признаков подвергается операции глобального усредняющего пулинга. Этот математический шаг схлопывает пространственные измерения матриц, вычисляя среднее арифметическое по каждой карте признаков, что делает алгоритм инвариантным к локальному расположению микроструктур на исходном снимке и формирует плотный одномерный вектор. В финале этот вектор умножается на матрицу весов единственного линейного терминального слоя, который агрегирует всю накопленную по ходу четырех макроэтапов семантическую информацию и проецирует ее в итоговое скалярное значение, отражающее предсказанный уровень влажности образца торфа.

1.3 Выбор программной среды и языка программирования

Разработка высоконагруженных интеллектуальных систем, базирующихся на алгоритмах компьютерного зрения, требует применения гибкого, производительного и масштабируемого программного инструментария. В качестве фундаментальной основы для программной реализации разрабатываемой системы был выбран высокоуровневый интерпретируемый язык программирования *Python*. На сегодняшний день данный язык заслуженно носит статус негласного мирового стандарта в индустрии анализа данных и машинного обучения. Это доминирующее положение продиктовано не только лаконичным и выразительным синтаксисом, который минимизирует время от возникновения идеи до создания работающего прототипа, но и беспрецедентно широкой экосистемой узкоспециализированных библиотек. Наличие мощных инструментов для векторизованных вычислений, математической статистики и работы с многомерными массивами данных позволяет разработчику абстрагироваться от низкоуровневого управления

памятью и сосредоточить все вычислительные ресурсы на реализации сложной математической логики регрессионного анализа.

При выборе основного языка разработки проводился тщательный сравнительный анализ *Python* с такими классическими компилируемыми языками индустриального стандарта, как *C++* и *Java*. Безусловно, языки с жесткой статической типизацией обладают определенным преимуществом в чистой скорости исполнения скомпилированного машинного кода. Однако в контексте глубокого машинного обучения это преимущество нивелируется, поскольку подавляющее большинство математических операций над тензорами все равно делегируется оптимизированным низкоуровневым библиотекам (написанным на *C/C++*), к которым языки высокого уровня обращаются через программные интерфейсы. При этом разработка архитектуры нейронной сети, конвейера предварительной обработки изображений и графического интерфейса на *C++* или *Java* требует написания огромного объема шаблонного кода. Громоздкий синтаксис и ограниченность специализированных фреймворков глубокого обучения делают традиционные языки неоптимальными для исследовательских задач. Ключевые критерии, предопределившие итоговый выбор инструментария, наглядно систематизированы и представлены в таблице 1.2.

Таблица 1.2 – Сравнительный анализ языков программирования

Язык программирования	Скорость исполнения кода	Развитость экосистемы машинного обучения	Сложность синтаксиса	Скорость прототипирования
<i>C++</i>	Очень высокая	Ограниченная	Высокая	Низкая
<i>Java</i>	Высокая	Умеренная	Средняя	Средняя
<i>Python</i>	Средняя	Максимальная	Низкая	Очень высокая

Определившись с базовым языком программирования, следующим критическим шагом этапа проектирования стал выбор фреймворка глубокого обучения, который возьмет на себя реализацию вычислительного ядра системы. На современном этапе развития технологий искусственного интеллекта индустриальными лидерами в этой области являются программные комплексы *TensorFlow* и *PyTorch*. Фреймворк *TensorFlow* исторически опирался на парадигму статического вычислительного графа, при которой разработчик сначала должен был полностью описать математическую архитектуру сети, скомпилировать ее в неизменяемый граф, и лишь затем запускать процесс передачи данных. Такой подход обеспечивал высокую степень оптимизации для промышленного развертывания,

однако делал процесс написания кода неинтуитивным, а отладку промежуточных тензорных операций – крайне затруднительной. Несмотря на то, что в последних версиях *TensorFlow* был внедрен режим динамических вычислений, архитектура фреймворка сохранила существенную громоздкость и перегруженность абстракциями.

Фундаментальным преимуществом библиотеки *PyTorch*, предопределившим ее безоговорочный выбор для реализации разрабатываемого программного обеспечения, является использование интуитивно понятного динамического вычислительного графа. В отличие от статических графов, в *PyTorch* вычислительный граф формируется динамически, «на лету», в момент выполнения каждой конкретной строки кода. Это позволяет разработчику взаимодействовать с многомерными тензорами точно так же, как с обычными массивами в стандартном языке *Python*, беспрепятственно извлекая и анализируя промежуточные значения градиентов на любом этапе прямого или обратного прохода нейросети. Интуитивная понятность, максимальная прозрачность процесса отладки и строгая приверженность идеологии объектно-ориентированного программирования делают *PyTorch* идеальным инструментом для тонкой настройки регрессионной архитектуры *ResNet34* и разработки кастомизированных функций потерь [4].

Неотъемлемым и критически важным аспектом работы с глубокими сверточными сетями является обеспечение приемлемой скорости выполнения миллионов матричных умножений. Выполнение подобных тензорных операций исключительно на центральном процессоре (*CPU*) делает процесс обучения и логического вывода катастрофически медленным. Традиционно эта проблема решается путем трансляции вычислений на графические ускорители через технологию *CUDA*. Однако использование *CUDA* накладывает жесткие аппаратные ограничения, так как эта архитектура является проприетарной и поддерживается исключительно видеокартами корпорации *NVIDIA*. Для преодоления этого фундаментального ограничения и обеспечения максимальной гибкости конечного продукта, в архитектуру разрабатываемой системы был интегрирован программный модуль *torch-directml*. Данная технология выступает в роли моста между высокоуровневым кодом *PyTorch* и низкоуровневым графическим *API DirectX 12*. Интеграция *DirectML* обеспечивает полноценное кросс-аппаратное ускорение вычислений, позволяя алгоритму задействовать вычислительные мощности не только тензорных ядер *NVIDIA*, но и графических процессоров (*GPU*) архитектур *AMD* и *Intel*. Применение такого подхода делает разрабатываемое программное обеспечение по-настоящему универсальным, независимым от конкретного поставщика оборудования и полностью готовым к развертыванию на широком спектре производственных вычислительных комплексов.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Выбор оборудования и условия получения датасета

Выбор аппаратного обеспечения для фотографической фиксации образцов торфа является критически важным этапом, который напрямую предопределяет качество входных тензоров для глубокой нейронной сети. Оптическое разрешение, динамический диапазон сенсора и механическая стабильность системы фундаментально влияют на способность регрессионной модели улавливать нелинейные зависимости между цветом, текстурой и искомым уровнем влажности. В качестве главного устройства для формирования обучающего набора данных была выбрана цифровая зеркальная камера с полупрозрачным зеркалом *Sony Alpha 58 (SLT-A58)*, относящаяся к классу продвинутых полупрофессиональных аппаратов, внешний вид которой представлен на рисунке 2.1. Данная модель оснащена высококачественной матрицей формата *APS-C* с физическим разрешением около 20 мегапикселей. Подобный размер сенсора обеспечивает оптимальный баланс между плотностью пикселей и их светочувствительностью, гарантируя захват мельчайших деталей исследуемой поверхности с минимальным уровнем цифрового шума, что крайне важно для алгоритмов выделения признаков на начальных сверточных слоях нейросети.



Рисунок 2.1 – Цифровая зеркальная камера *Sony Alpha 58*

Учитывая специфику съемочного процесса, требовавшего строгой пространственной фиксации кадров, неоспоримым эргономическим преимуществом камеры *Sony Alpha 58* стало наличие поворотного

жидкокристаллического дисплея. Этот элемент конструкции существенно упростил процесс кадрирования и точной фокусировки при вертикальном расположении объектива непосредственно над лотками с сырьем. Однако съемка с высоким разрешением неизбежно многократно усиливает негативное влияние любых механических вибраций. Для полного исключения риска возникновения оптических микросмазов, которые необратимо разрушают высокочастотную текстурную информацию на пиксельном уровне, камера была жестко закреплена на массивном профессиональном штативе, как показано на рисунке 2.2. Использование штативной установки не только обеспечило абсолютную резкость каждого кадра, но и позволило стандартизировать фокусное расстояние и угол наклона оптической оси для всей выборки, сохранив геометрическую однородность входных данных, что является фундаментальным требованием при подготовке датасетов для сверточных сетей.



Рисунок 2.2 – Использование штативной установки для механической стабилизации камеры

Для извлечения базовых визуальных маркеров влажности, таких как капиллярное набухание органических волокон и степень адгезии частиц торфа, стандартного оптического увеличения оказалось недостаточно. Вместо использования алгоритмов цифрового масштабирования, приводящих к интерполяционному размытию, оптическая система камеры была

модифицирована путем установки специализированных макроколец, представленных на рисунке 2.3. Интеграция данных удлинительных колец между байонетом камеры и объективом позволила радикально сократить минимальную дистанцию фокусировки, превратив стандартный объектив в мощный инструмент макросъемки. Подобная оптическая конфигурация обеспечила захват глубоких микроструктурных характеристик материала на физическом уровне. Для архитектуры *ResNet34* такая микроскопическая детализация имеет первостепенное значение, поскольку визуальный переход торфа из сухой, рыхлой фазы в плотную, переувлажненную массу проявляется именно на уровне микропор и слияния отдельных структурных агрегатов, которые невозможно зафиксировать при общих планах съемки.



Рисунок 2.3 – Оптические удлинительные макрокольца для масштабирования структуры торфа

Фундаментальной оптической проблемой при оценке влажности высокопористых материалов является эффект зеркального отражения. По мере приближения уровня влажности к верхним границам диапазона (2400 – 2650 условных единиц), на поверхности торфа формируются микроскопические пленки свободной воды, которые начинают интенсивно отражать направленный свет. Этот физический процесс приводит к появлению критических засветов («выбитых» белых пикселей) на матрице, которые ослепляют сенсор и полностью скрывают под собой истинную текстуру сырья. Для подавления этого деструктивного оптического явления на фронтальную линзу объектива был установлен циркулярный поляризационный фильтр (*CPL*), внешний вид которого продемонстрирован на рисунке 2.4.



Рисунок 2.4 – Циркулярный поляризационный фильтр для устранения водных бликов

Вращение поляризатора позволило избирательно отсечь поляризованные световые волны, ответственные за избыточные блики от водной поверхности. Такое оптическое вмешательство дало возможность камере «просмотреть» сквозь бликующую пленку, выявив для нейросети истинную глубинную текстуру и подлинную цветовую насыщенность переувлажненного торфа, являющиеся самыми надежными предикторами целевой переменной.

2.2 Формирование датасета и алгоритмы предварительной обработки данных

Создание надежной и физически обоснованной предсказательной модели машинного обучения требует формирования строго выверенного набора данных, отражающего реальную динамику изменения состояния исследуемого материала. С этой целью в рамках работы был проведен специализированный лабораторный эксперимент по искусственному моделированию различных уровней увлажнения торфа. В качестве инструментального базиса использовались стандартизированные емкости для образцов, в которые помещались равные по объему пробы абсолютно сухого торфа. Для контролируемого изменения физико-химического состояния сырья применялся метод строго дозированного увлажнения. С помощью мерного медицинского шприца в подготовленные сухие образцы последовательно вводилась вода в строго заданных объемах: 0 миллилитров (контрольный сухой образец), 1 миллилитр, 2 миллилитра, 3 миллилитра, 5 миллилитров, 7 миллилитров и 9 миллилитров.

Поскольку торф обладает сложной капиллярно-пористой структурой, простое добавление жидкости неизбежно приводит к образованию локальных очагов переувлажнения на поверхности при сохранении сухой фракции на дне емкости. Для устранения этой физической неоднородности после каждой инъекции заданного объема воды материал подвергался длительному и

тщательному механическому перемешиванию. Данная процедура позволила разрушить образовавшиеся комки, добиться равномерного распределения молекул воды по всему объему образца и симитировать естественное состояние гомогенного увлажненного сырья, с которым алгоритмам компьютерного зрения предстоит сталкиваться в реальных производственных условиях.

Для перевода визуальных изменений в объективный математический формат и создания обучающих меток требовалось получить эталонные числовые значения для каждого подготовленного образца. Оценка производилась с применением электронного емкостного датчика влажности, принцип действия которого основан на измерении диэлектрической проницаемости среды. Чувствительный щуп датчика погружался в центр каждого тщательно перемешанного образца, после чего стабильные показания встроенного аналого-цифрового преобразователя фиксировались в базу данных. Результаты физического эксперимента продемонстрировали четкую зависимость: контрольному сухому образцу, в который не добавлялась вода (0 мл), соответствовало базовое значение в 1720 условных единиц, в то время как максимально увлажненной пробе (9 мл) соответствовал пиковый показатель в 2650 единиц.

Таблица 2.1 – Соответствие добавленного объема воды показаниям емкостного датчика влажности

Добавленный объем воды, мл	Физическое состояние образца	Показания емкостного датчика, усл. ед
0	Сухой торф	1720
1	Слабое поверхностное увлажнение	1900
2	Умеренное локальное увлажнение	1920
3	Средняя степень увлажненности	2150
5	Повышенная влажность	2200
7	Сильная увлажненность	2400
9	Переувлажненная масса	2650

Проведенное дозированное увлажнение спровоцировало не только изменение электрических характеристик, но и закономерную трансформацию оптических свойств поверхности торфа. По мере увеличения объема введенной воды наблюдалось выраженное потемнение цветового тона и укрупнение микроструктуры за счет капиллярного слипания мелких частиц. В результате

лабораторного эксперимента была сформирована дискретная шкала, состоящая из семи уникальных физических классов влажности, каждый из которых обладает собственным набором специфических визуальных паттернов, зафиксированных цифровой камерой. Визуальные представители каждого из 7 сформированных классов влажности (самый сухой – слева, самый влажный – справа) представлены на рисунке 2.5.



Рисунок 2.5 – Визуальные представители каждого из 7 сформированных классов влажности

Исходная база визуальных материалов представляет собой структурированный архив папок, содержащих цифровые фотографии поверхности торфа высокого разрешения. Каждой директории в этой файловой иерархии строго поставлено в соответствие конкретное физическое значение влажности материала, полученное объективными аппаратными методами. Выборка охватывает широкий диапазон значений и состоит из семи верифицированных дискретных уровней влажности: 1720, 1900, 1920, 2150, 2200, 2400 и 2650 условных единиц.

Для обеспечения чистоты и репрезентативности обучающей выборки был спроектирован и программно реализован специализированный многоступенчатый конвейер предварительной обработки изображений, общая блок-схема которого представлена на рисунке 2.2. Первым этапом данного конвейера является алгоритм автоматической отбраковки визуально неполноценных снимков. Программная реализация данного модуля выполнена с использованием библиотеки обработки изображений *Pillow (Python Imaging Library)*. В процессе первичного анализа программа переводит каждое входное изображение в цветовое пространство градаций серого, после чего посредством встроенного класса *ImageStat* математически вычисляет глобальные статистические метрики распределения пикселей, такие как средняя яркость и среднеквадратичное отклонение, отражающее общую контрастность кадра. Опираясь на заранее откалиброванные пороговые значения этих метрик, алгоритм надежно отсеивает слишком темные, критически засвеченные или неконтрастные (размытые) кадры, полностью исключая попадание зашумленной или неинформативной визуальной информации в тренировочный датасет.

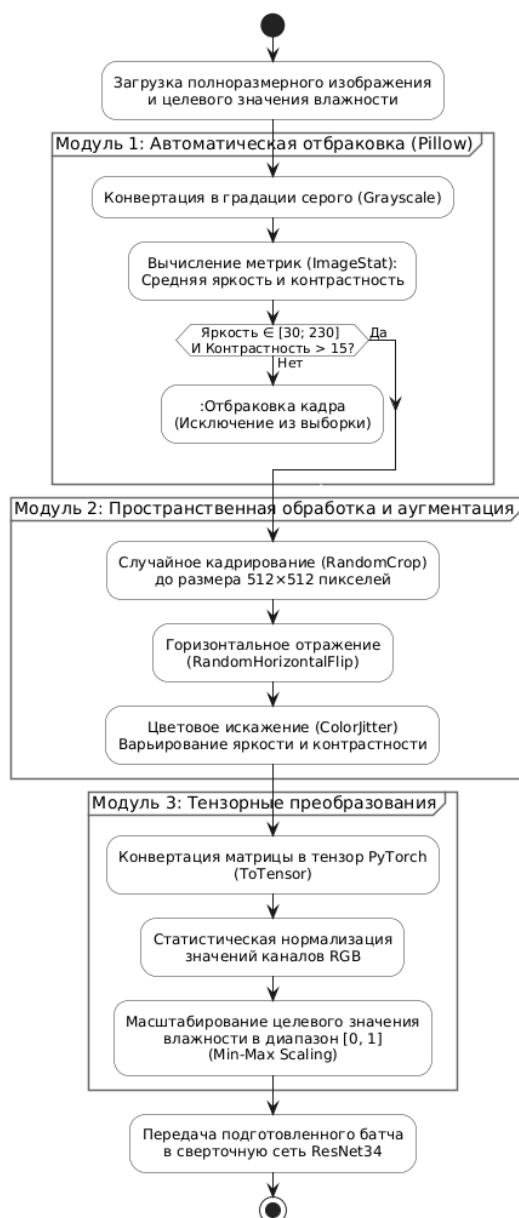


Рисунок 2.6 – Блок-схема программного конвейера обработки изображений

После успешного прохождения автоматической фильтрации дефектных изображений выполняется пространственно-цветовая подготовка утвержденных кадров. Поскольку исходные фотографии обладают избыточно высоким разрешением, прямое сжатие таких изображений до стандартных размеров тензорного входа нейронной сети неизбежно привело бы к безвозвратной потере тонких оптических характеристик. Для гарантированного сохранения детализированной микроструктуры торфа применяется программный метод случайного кадрирования (*RandomCrop*). В ходе выполнения данного алгоритма из каждого полноразмерного валидного снимка динамически извлекаются квадратные области строго установленной размерности 512×512 пикселей.

С целью повышения робастности нейросетевой модели и предотвращения переобучения, базовое кадрирование интегрировано в комплексный вычислительный конвейер аугментации данных. Помимо стохастического смещения координат вырезаемых областей, к изображениям применяются алгоритмы горизонтального отражения, а также метод цветового искажения (*ColorJitter*), вносящий контролируемые случайные флуктуации в исходные параметры яркости и контрастности снимка. Подобный подход многократно увеличивает общий объем и вариативность обучающей выборки. Финальным этапом подготовки входных данных является преобразование матриц изображений в формат многомерных тензоров с их последующей статистической нормализацией, что обеспечивает приведение распределения пиксельных значений к стандартизированному виду, оптимальному для обработки сверточными слоями.

Завершающим шагом конвейера предварительной подготовки данных выступает обязательная трансформация целевой переменной. Необработанные физические значения влажности обладают большим порядком величины, что оказывает деструктивное влияние на вычислительную стабильность в процессе обучения искусственной нейронной сети. Существует строгая математическая необходимость масштабирования этих целевых значений в нормализованный диапазон от нуля до единицы. Если алгоритм вынужден предсказывать большие абсолютные числа, вычисляемая ошибка пропорционально возрастает, что неизбежно приводит к генерации чрезмерно высоких значений градиентов на этапе обратного распространения ошибки. Это явление вызывает резкие осцилляции весовых коэффициентов и потенциальную расходимость функции потерь. Строгая линейная нормализация целевого параметра обеспечивает соразмерность выходных сигналов модели с внутренними параметрами сети, гарантируя плавную и стабильную сходимость алгоритма оптимизации к оптимальному состоянию.

2.3 Проектирование регрессионной модели и процесса обучения

Адаптация предварительно обученной архитектуры глубокой сверточной нейронной сети *ResNet34* для решения задачи регрессии требует проведения точечной программной модификации ее топологии. В исходном состоянии данная модель алгоритмически ориентирована на многоклассовую классификацию изображений. Для интеграции в разрабатываемую систему терминальный полносвязный слой классификатора (*fc*) программно удаляется и заменяется на новый линейный слой (*nn.Linear*), сконфигурированный на наличие ровно одного выходного. Особенностью является инициализация весовых параметров нового слоя. Поскольку целевые значения влажности предварительно масштабируются с использованием метода минимаксной нормализации (*Min-Max Scaling*) в строгий математический диапазон $[0, 1]$, базовое смещение (*bias*) добавленного

полносвязного слоя принудительно инициализируется константным значением 0,5. Установка начального смещения в математическое ожидание нормализованного интервала позволяет сети сразу генерировать адекватные первичные предсказания, существенно сокращая время поиска оптимальных весовых коэффициентов и повышая общую вычислительную стабильность на старте обучения.

В качестве объективной целевой метрики расхождения между истинными значениями влажности и ответами, генерируемыми нейронной сетью, интегрирована функция потерь *L1Loss*, математически эквивалентная средней абсолютной ошибке (*Mean Absolute Error, MAE*). Выбор данного критерия оптимизации напрямую обусловлен его высокой робастностью к возможным статистическим выбросам и аномалиям в обучающей выборке, которые могут возникать из-за естественной оптической неоднородности микроструктуры торфа. В отличие от среднеквадратичной функции потерь, чрезмерно штрафующей сеть за единичные сильные отклонения, средняя абсолютная ошибка генерирует равномерный градиент на всем протяжении процесса обучения. Использование *L1Loss* заставляет алгоритм точнее и плавнее подстраивать внутренние веса, минимизируя прямую линейную разницу между вычисленным и эталонным значениями, что способствует формированию высокоустойчивой регрессионной модели.

Процесс минимизации вычисленной ошибки осуществляется посредством современного оптимизатора *AdamW* с начальным шагом обучения, установленным на уровне $1e-4$. Данный алгоритм представляет собой усовершенствованную модификацию метода адаптивной оценки моментов с корректно реализованным механизмом регуляризации весов, что эффективно предотвращает переобучение модели за счет обособленного применения штрафа за чрезмерно большие значения весовых коэффициентов. Динамический контроль над градиентным спуском обеспечивается программным планировщиком *ReduceLROnPlateau*. Данный механизм автоматически снижает скорость обновления весов (умножая шаг на коэффициент 0,5) при обнаружении стагнации функции потерь на валидационной выборке в течение заданного числа эпох, полностью исключая риск расходимости в области локального минимума.

Критически важным аспектом программной реализации является обеспечение высокой производительности тензорных вычислений. Для минимизации времени тренировочного цикла в программный комплекс внедрена кросс-платформенная поддержка аппаратного ускорения с использованием технологии *torch-directml* (*Direct Machine Learning*). Данное архитектурное решение позволяет эффективно задействовать вычислительные мощности графических процессоров (*GPU*) от *AMD* и *Intel*, наряду с классической поддержкой аппаратной среды *NVIDIA CUDA*, что делает спроектированную предсказательную систему полностью аппаратно-независимой.

Наконец, для обеспечения прозрачного мониторинга процесса обучения, метрики ошибки, вычисляемые ядром на нормализованных данных, динамически подвергаются обратному алгебраическому преобразованию. Вычисление по формуле $Error * (Target_max - Target_min)$ позволяет выводить в консоль и на графические элементы пользовательского интерфейса интуитивно понятные значения абсолютной ошибки непосредственно в исходных физических единицах измерения влажности по мере завершения каждой тренировочной эпохи.

2.4 Разработка графического интерфейса и модуля предсказания

Создание эргономичного программного продукта требует тщательного визуального проектирования. Перед написанием кода графической оболочки был проведен анализ требований к интерфейсу оператора и разработаны первичные каркасные эскизы. Метод каркасного моделирования позволил абстрагироваться от визуального оформления и сфокусироваться на информационной архитектуре, распределении функциональных зон и логике взаимодействия. Главной задачей этапа выступало создание среды, позволяющей неподготовленному оператору интуитивно управлять сложными тензорными вычислениями, не вникая в математическую структуру нейронной сети. Эскиз главного окна приложения для выбора и анализа фотографий представлен на рисунке 2.7.

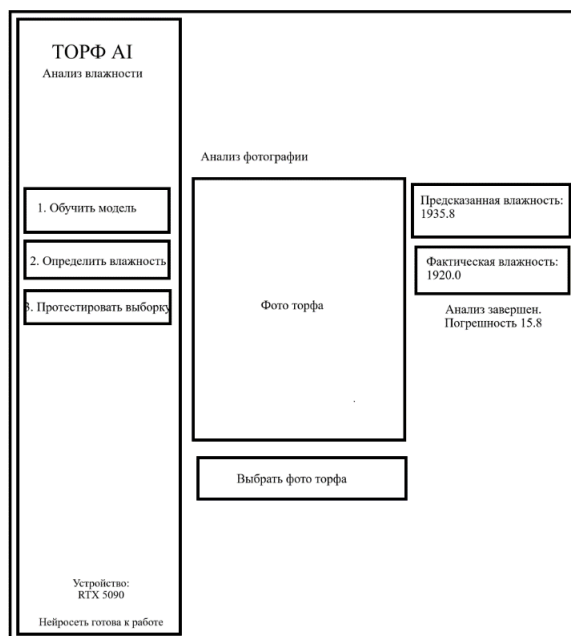


Рисунок 2.7 – Эскиз главного окна приложения

Опираясь на утвержденный эскиз, логика расположения графических элементов была выстроена по классическому принципу двухпанельной компоновки. Левая вертикальная панель была выделена под глобальную

навигацию и статусную информацию. На ней сгруппированы массивные управляющие кнопки, отвечающие за инициализацию трех фундаментальных режимов работы системы: обучение модели, анализ одиночных изображений и тестирование на отложенной выборке. Здесь же размещен динамический блок, отображающий текущий статус загрузки весов нейросети и тип активированного аппаратного ускорителя (*CPU* или *GPU*). Подобное структурное решение гарантирует, что ключевые элементы управления всегда остаются в поле зрения пользователя, независимо от текущего контекста работы.

Правая, более объемная часть окна, была спроектирована как динамическая рабочая область. Ее структура адаптивно изменяется в зависимости от выбранного режима навигационной панели. В контексте основного модуля предсказания рабочая область была визуальным разделена на зону загрузки и отображения миниатюры исследуемой фотографии и зону вывода числовых метрик. Для повышения когнитивной легкости восприятия информации, итоговые математические результаты расчета регрессионной модели («Предсказанная влажность») и показатели абсолютной ошибки выводятся крупным контрастным шрифтом в виде обособленных информационных карточек. Спроектированная концептуальная модель была успешно транслирована в программный код, обеспечив создание целостного инструментария.

Практическая реализация разработанного алгоритмического комплекса выполнена в виде полноценного оконного приложения на базе встроенной графической библиотеки *Tkinter*, предоставляющего оператору интуитивно понятный инструментарий для управления процессами машинного обучения и визуального анализа новых данных. Главный экран программы спроектирован таким образом, чтобы обеспечить удобный и быстрый доступ к трем фундаментальным функциям: инициализации цикла обучения нейронной сети, запуску диагностического модуля для оценки произвольных фотографий и тестированию модели на отложенной выборке. Особое внимание при проектировании программного обеспечения было уделено архитектурному решению проблемы возможной блокировки пользовательского окна во время выполнения длительных математических операций. Поскольку процесс вычисления градиентов, обратного распространения ошибки или пакетного прогона тензоров через глубокую сверточную сеть требует колоссальных вычислительных ресурсов, запуск этих процедур в основном системном потоке неизбежно приводил бы к полному зависанию графической оболочки. Для решения этой критической проблемы применена многопоточная архитектура приложения с использованием стандартного модуля *threading*, при которой тяжелые тензорные вычисления физически изолируются и асинхронно выполняются в выделенном фоновом потоке. Данный подход гарантирует непрерывную отзывчивость графического интерфейса *Tkinter*,

позволяя оператору отслеживать статус выполнения задач через динамические индикаторы и сохранять полный контроль над приложением [5].

Ключевым вычислительным компонентом созданной программы выступает модуль автоматического предсказания. При инициировании оператором процесса оценки новой фотографии алгоритм применяет стратегию множественного локального анализа, алгоритмически представляющую собой ансамблирование на этапе логического вывода. Опираясь на принципы сохранения микроструктуры, заложенные на этапе подготовки данных, вычислительное ядро в автоматическом режиме стохастически вырезает из загруженного снимка ровно пять независимых квадратных фрагментов размером 512 на 512 пикселей.

Каждый из сформированных сегментов преобразуется в нормализованный тензор и независимо пропускается через слои обученной нейронной сети, генерируя массив из пяти автономных предсказаний. На завершающем этапе математической обработки программа вычисляет точное среднее арифметическое полученных значений, денормализует его, масштабируя обратно в реальные физические единицы, и выводит на экран в качестве итогового ответа. Подобное усреднение результатов одного кадра многократно повышает общую робастность и надежность системы к возможным локальным оптическим артефактам (пересвеченным участкам или теням от комков), эффективно нивелируя их деструктивное влияние. Визуальное оформление данного диагностического модуля наглядно представлено на рисунке Г.3.

Ссылка на удаленный репозиторий: <https://github.com/mystery-yogurt/PeatAI.git>

Полный листинг разработанной программы приведён в приложении А.

3 ВЕРИФИКАЦИЯ И ТЕСТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА

3.1 Анализ процесса обучения по графикам функции потерь

Первичным этапом оценки качества спроектированной нейросетевой модели является визуальный анализ динамики изменения ошибки в процессе тренировки. На рисунке 3.1 представлен график обучения, отражающий синхронное снижение средней абсолютной ошибки на обучающей и валидационной выборках на протяжении 20 расчетных эпох.

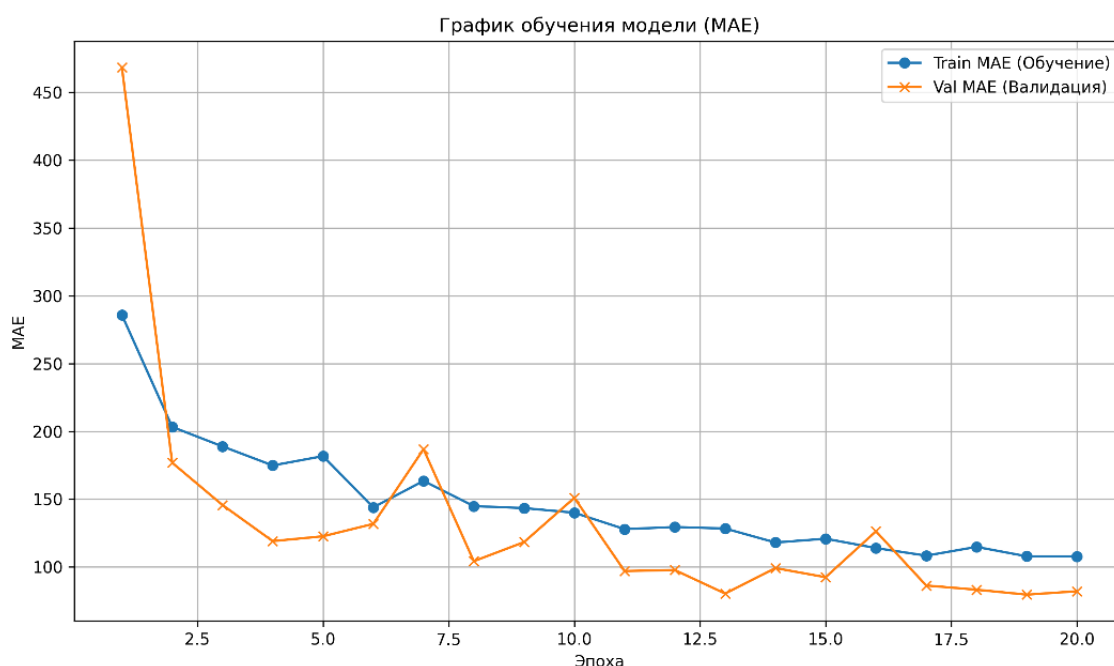


Рисунок 3.1 – График обучения модели

Плавное убывание обеих кривых без резких расхождений и осцилляций свидетельствует о корректной настройке шага обучения оптимизатора и правильной работе встроенного планировщика. Отсутствие значительного зазора между тренировочной и валидационной кривыми на завершающих этапах обучения доказывает отсутствие эффекта критического переобучения, что было успешно достигнуто благодаря строгой фильтрации исходных кадров и применению агрессивной аугментации данных. Выход кривых на горизонтальное плато подтверждает успешную сходимость алгоритма к глобальному оптимуму.

Для объективной проверки обобщающей способности и надежности разработанной нейросетевой системы был проведен этап финального тестирования на полностью изолированном наборе данных, который ранее не использовался в процессе оптимизации весовых коэффициентов. Анализ

производительности алгоритма осуществлялся с применением комплекса стандартизированных математических метрик, позволяющих всесторонне оценить качество решения задачи регрессии. Результаты прогона тестовой выборки продемонстрировали высокую точность обученной модели, что подтверждается полученными числовыми показателями вычисляемых ошибок.

Анализ абсолютных отклонений показал, что средняя абсолютная ошибка (*MAE*), которая вычисляется по формуле (3.1), составила 78,7 условных единиц.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|; \quad (3.1)$$

где:

n – общее количество изображений в тестовой выборке;

y_i – истинное (эталонное) значение влажности i -го образца;

$\hat{y}_i$ – предсказанное значение влажности для i -го образца.

Учитывая широкий разброс целевых значений влажности в исходном датасете (от 1720 до 2650 единиц), данная величина отклонения является незначительной и свидетельствует о том, что сеть успешно уловила базовые оптические закономерности. Дополнительно был вычислен корень из среднеквадратичной ошибки (*RMSE*), расчет которого производится по формуле (3.2), значение которого зафиксировалось на отметке 99,8.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}; \quad (3.2)$$

Поскольку квадратичная функция потерь математически сильнее штрафует алгоритм за крупные промахи, сравнительно небольшая разница между показателями *RMSE* и *MAE* указывает на отсутствие критических аномальных выбросов в предсказаниях. Это означает, что модель работает стабильно и не допускает катастрофических ошибок даже на сложных или визуально зашумленных фрагментах торфа.

Особое практическое значение для оценки прикладного потенциала системы имеют относительные статистические показатели. Средняя абсолютная ошибка в процентах (*MAPE*), вычисляемая по формуле (3.3), по итогам тестирования составила всего 3,7%.

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|; \quad (3.3)$$

Столь низкий уровень относительной погрешности неопровержимо доказывает применимость разработанного программно-аппаратного комплекса в реальных производственных условиях, где требования к точности инструментальных измерений традиционно высоки. Дополнительным подтверждением эффективности выбранной архитектуры выступает коэффициент детерминации (R^2), определяемый по формуле (3.4), достигший значения 0,886.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (3.4)$$

где:

$\bar{y}_i$ – вычисленное среднее арифметическое истинных значений влажности тестовой выборки.

Данный показатель математически отражает долю дисперсии зависимой переменной, которую способна объяснить модель. Значение R^2 , вплотную приближенное к отметке 0,9, говорит о крайне высокой предсказательной способности системы и наличии строгой линейной зависимости между фактическими и предсказанными нейросетью уровнями влажности материала [6].

Сводный экран диагностического модуля разработанного графического интерфейса, наглядно отображающий результаты автоматизированного расчета описанных выше метрик качества после успешного завершения обработки всей тестовой выборки, представлен на рисунке Г.4.

3.2 Оценка качества предсказаний и тестирование интерфейса

Для более глубокого понимания характера работы обученной регрессионной модели и выявления возможных скрытых закономерностей в ее ответах был проведен детальный визуальный анализ сгенерированных графических отчетов. На диаграмме рассеяния, отражающей соотношение истинной и предсказанной влажности, отчетливо видно, что эмпирические точки плотно группируются вдоль красной диагональной линии идеального совпадения. Такая конфигурация кластера подтверждает высокую степень линейности предсказаний и доказывает способность нейросети корректно аппроксимировать целевую функцию на всем исследуемом диапазоне. В свою очередь, гистограмма распределения абсолютных ошибок демонстрирует форму, визуально и статистически близкую к классическому нормальному (гауссовскому) распределению с явно выраженным математическим ожиданием (пиком) в окрестности нуля. Подобная симметричная структура распределения свидетельствует об отсутствии у модели системного

смещения: вычислительный алгоритм не склонен к постоянному завышению или занижению результатов, а возникающие погрешности носят преимущественно случайный характер. Графическое представление описанных метрических зависимостей приведено на рисунке 3.2.

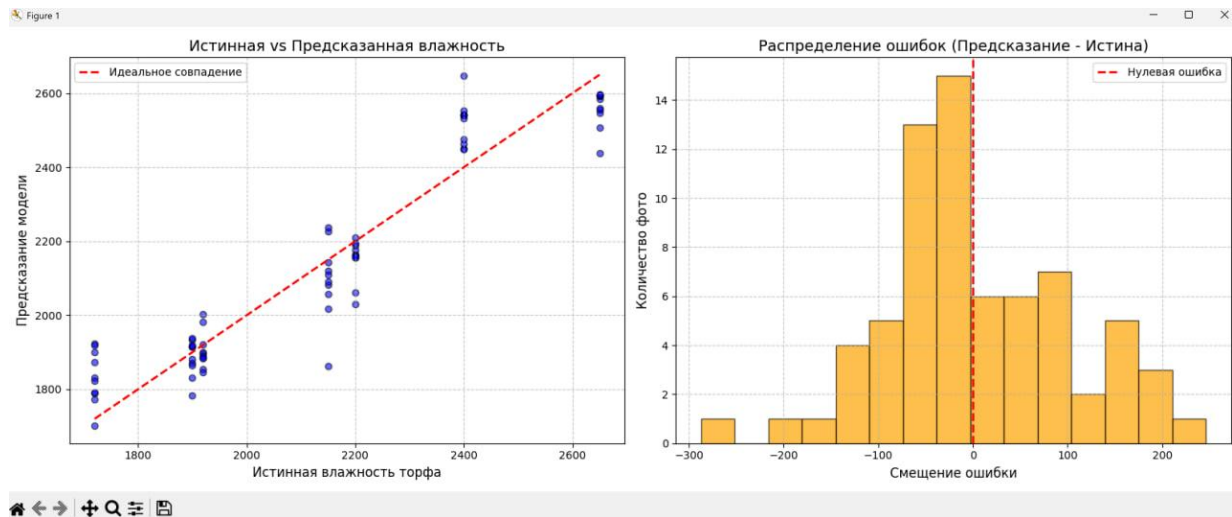


Рисунок 3.2 – Диаграмма рассеяния и диаграмма ошибок

Важным этапом верификации стала практическая оценка точности алгоритма на конкретных визуальных образцах сырья, представленных в виде сгенерированной сводной сетки предсказаний по всем семи верифицированным классам влажности. Результаты покадрового анализа и визуального сравнения локальных предсказаний с эталонными показателями наглядно продемонстрированы на рисунке 3.3.



Рисунок 3.3 – Сравнения предсказаний с эталонными показателями влажности

Для оценки внутренней логики обученной нейросети и визуализации выученной функции регрессии был проведен анализ латентного пространства признаков. Архитектура *ResNet34* извлекает из каждого изображения высокоразмерный вектор из 512 уникальных оптических характеристик. Чтобы спроецировать этот массив на плоскость, был применен метод анализа главных компонент (PCA). С его помощью 512 признаков были математически сжаты до единственного доминирующего вектора, объединяющего данные о яркости, текстуре и бликах. Графическое отображение работы алгоритма в координатах этого комплексного признака представлено на рисунке 3.4.

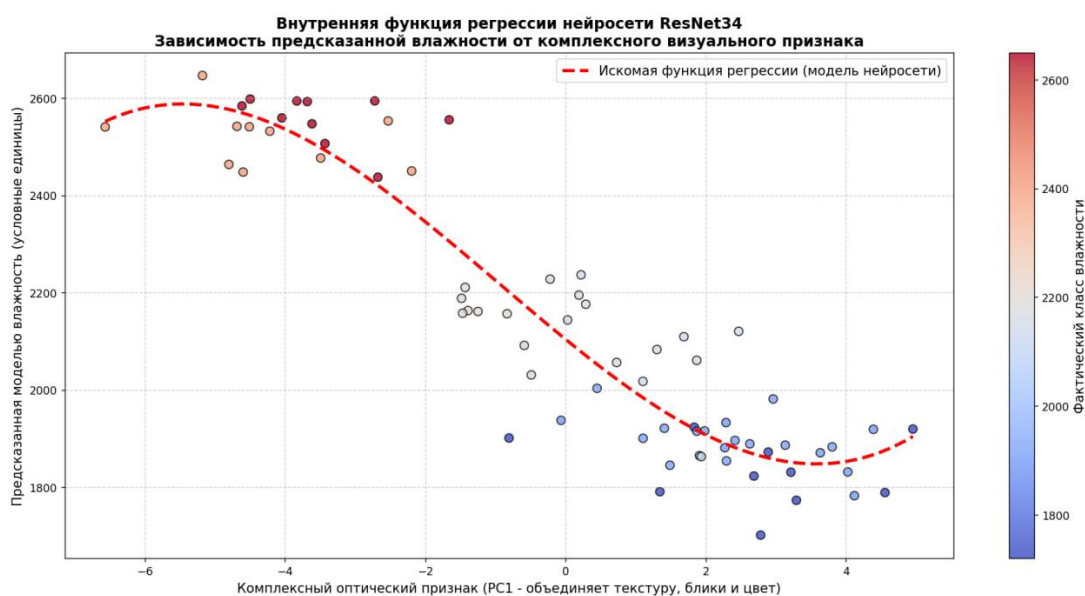


Рисунок 3.4 – Внутренняя функция регрессии нейросети *ResNet34*

Ось абсцисс на графике отражает вычисленный комплексный признак, а ось ординат – предсказанную влажность. Цветовая дифференциация точек от синего к красному строго соответствует фактическому аппаратному классу влажности. На графике отчетливо виден плавный градиентный переход от сухих образцов к переувлажненным, что доказывает способность сети выстраивать физически обоснованную шкалу состояний сырья.

Красная пунктирная линия представляет собой двумерную геометрическую проекцию построенной гиперплоскости регрессии. Ее выраженный нелинейный характер обусловлен физикой капиллярного впитывания: первоначальное резкое потемнение торфа со временем сменяется появлением поверхностных зеркальных пленок воды, что закономерно сглаживает кривую на этапе насыщения. Таким образом, анализ подтверждает, что нейросеть успешно синтезировала непрерывную математическую функцию, способную интерполировать промежуточные состояния материала, функционируя как высокоточный виртуальный измерительный прибор.

ЗАКЛЮЧЕНИЕ

Итогом курсовой работы стала разработка интеллектуального программного комплекса с графическим интерфейсом для автоматического предсказания уровня влажности торфа по цифровым фотографиям. Созданный продукт скрывает сложные тензорные вычисления за удобной пользовательской оболочкой и обеспечивает высокую надежность визуального анализа природных материалов.

Фундаментальным преимуществом системы является применение регрессионной нейросетевой архитектуры, что позволило оценивать влажность по непрерывной шкале и отказаться от жесткой классификации состояний сырья. Высокая точность алгоритма обеспечена автоматической фильтрацией дефектных (засвеченных и размытых) снимков на этапе предобработки, а также методом усреднения предсказаний по нескольким случайно вырезанным фрагментам кадра, что эффективно нивелирует влияние оптических артефактов.

Тестирование разработанной модели на независимой выборке подтвердило высокую эффективность подхода: средняя абсолютная ошибка (MAE) составила 78,7 единиц, средняя абсолютная процентная ошибка ($MAPE$) – 3,7%, а коэффициент детерминации (R^2) достиг 0,886. Данные метрики неопровержимо доказывают, что модель успешно выявила нелинейные визуальные закономерности и пригодна для эксплуатации.

Практическая значимость проекта заключается в возможности внедрения разработанного решения в энергетический и агропромышленный секторы. Замена ресурсоемких лабораторных процедур на мгновенный бесконтактный визуальный анализ открывает перспективы для непрерывного автоматизированного мониторинга торфа на конвейерных линиях, что существенно снизит издержки и повысит эффективность переработки сырья.

Список использованных источников

1. Торфяная промышленность: Технологии и оборудование для контроля качества сырья – Электрон. данные. – Режим доступа: <https://mining-media.ru/ru/article/vagon/torf>. – Дата доступа: 02.04.2026.
2. Крамаренко В.В. и др. Обзор современных методов и технических средств измерения влажности торфа – Горный информационно-аналитический бюллетень. – 2021. – № 11-1. – С. 85–92.
3. Skye J. и др. PeatDepth-ML: Глобальная карта глубины торфа, полученная с помощью машинного обучения – EGUsphere. – 2025. DOI: <https://doi.org/0.594/egusphere-2025-5363>.
4. Paszke, A. и др. PyTorch: Библиотека глубокого обучения императивного стиля с высокой производительностью – Advances in Neural Information Processing Systems 32 (NeurIPS). – 2019.
5. Лутц М. «Программирование на Python, том 1» (Раздел по созданию GUI интерфейсов на Tkinter) – 4-е изд. – Санкт-Петербург: «Символ-Плюс», 2011. – 992 с.
6. Scikit-Learn regression metrics (Метрики регрессии в Scikit-Learn): Электрон. данные. – Режим доступа: https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics. – Дата доступа: 01.05.2026.
7. Ковалев А. В., Смирнов С. И. Нейросетевые методы регрессионного анализа в задачах оценки влажности дисперсных материалов // Вестник компьютерных и информационных технологий. – 2022. – № 8. – С. 45–52.
8. Хайден Ю., Мирджалили В. Машинное обучение с помощью PyTorch и Scikit-Learn. – Санкт-Петербург: Диалектика, 2023. – 850 с.
9. Шоле Ф. Глубокое обучение на Python. – 2-е изд. – Санкт-Петербург: Питер, 2022. – 576 с.
10. Torchvision Models and pre-trained weights (Архитектуры сверточных сетей семейства ResNet): Электрон. данные. – Режим доступа: <https://pytorch.org/vision/stable/models.html>. – Дата доступа: 15.04.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы

app.py:

```
import os
import threading
import time
import random
import numpy as np
import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image, ImageTk
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, mean_absolute_percentage_error
import matplotlib.pyplot as plt
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from PIL import ImageStat
from typing import List, Tuple, Dict, Optional
import warnings

warnings.filterwarnings('ignore')

try:
    import torch_directml
    HAS_DML = True
except ImportError:
    HAS_DML = False

TARGET_MAPPING = {
    "1": 1720.0, "2": 1900.0, "3": 1920.0,
    "4": 2150.0, "5": 2200.0, "6": 2400.0, "7": 2650.0
}
TARGET_MIN, TARGET_MAX = 1720.0, 2650.0
MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]

MODEL_PATH = "best_peat_model.pth"
TEST_DIR = "dataset_test"

BG_COLOR = "#F4F7FB"
PANEL_BG = "#FFFFFF"
PRIMARY = "#4A90E2"
SUCCESS = "#2ECC71"
WARNING = "#F39C12"
INFO = "#8E44AD"
TEXT_MAIN = "#2C3E50"
TEXT_MUTED = "#7F8C8D"

def is_good_image(img_path: str) -> bool:
    try:
        with Image.open(img_path) as img:
            img_gray = img.convert('L')
            stat = ImageStat.Stat(img_gray)
            brightness = stat.mean[0]
```

```

        contrast = stat.stddev[0]
        if brightness < 30: return False
        if brightness > 230: return False
        if contrast < 15: return False
    return True
except Exception:
    return False

class PeatDataset(Dataset):
    def __init__(self, root_dir: str, transform: Optional[transforms.Compose] = None, filter_bad: bool = False):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths: List[str] = []
        self.targets: List[float] = []

        if not os.path.exists(root_dir):
            raise FileNotFoundError(f"Директория {root_dir} не найдена!")

        for folder_name in os.listdir(root_dir):
            if folder_name in TARGET_MAPPING:
                folder_path = os.path.join(root_dir, folder_name)
                if os.path.isdir(folder_path):
                    target_value = TARGET_MAPPING[folder_name]
                    target_scaled = (target_value - TARGET_MIN) / (TARGET_MAX - TARGET_MIN)

                    for file_name in os.listdir(folder_path):
                        if file_name.lower().endswith(('.jpg', '.jpeg', '.png')):
                            full_path = os.path.join(folder_path, file_name)
                            if filter_bad and not is_good_image(full_path):
                                continue
                            self.image_paths.append(full_path)
                            self.targets.append(target_scaled)

    def __len__(self) -> int:
        return len(self.image_paths)

    def __getitem__(self, idx: int) -> Tuple[torch.Tensor, torch.Tensor]:
        image = Image.open(self.image_paths[idx]).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, torch.tensor([self.targets[idx]], dtype=torch.float32)

def create_data loaders(root_dir: str, batch_size: int = 8, val_split: float = 0.2) -> Tuple[DataLoader, DataLoader]:
    train_transform = transforms.Compose([
        transforms.RandomCrop(512),
        transforms.RandomHorizontalFlip(),
        transforms.ColorJitter(brightness=0.1, contrast=0.1),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD)
    ])
    val_transform = transforms.Compose([
        transforms.CenterCrop(512),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD)
    ])

    full_dataset = PeatDataset(root_dir=root_dir, transform=None, filter_bad=True)
    dataset_size = len(full_dataset)
    indices = list(range(dataset_size))
    random.shuffle(indices)
    split = int(torch.floor(torch.tensor(val_split * dataset_size)))

```

```

train_indices, val_indices = indices[split:], indices[:split]

train_dataset = PeatDataset(root_dir=root_dir, transform=train_transform, filter_bad=False)
val_dataset = PeatDataset(root_dir=root_dir, transform=val_transform, filter_bad=False)

train_dataset.image_paths = [full_dataset.image_paths[i] for i in train_indices]
train_dataset.targets = [full_dataset.targets[i] for i in train_indices]
val_dataset.image_paths = [full_dataset.image_paths[i] for i in val_indices]
val_dataset.targets = [full_dataset.targets[i] for i in val_indices]

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0,
pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=0, pin_memory=True)

return train_loader, val_loader

class DashboardApp:
    def __init__(self, root):
        self.root = root
        self.root.title("ТОПФ AI - Интеллектуальный анализ торфа")
        self.root.geometry("1000x650")
        self.root.configure(bg=BG_COLOR)

        self.model = None

        if torch.cuda.is_available():
            self.device = torch.device("cuda")
            self.device_name = f"NVIDIA CUDA ({torch.cuda.get_device_name(0)})"
        elif HAS_DML and torch_directml.is_available():
            dml_device_id = torch_directml.default_device()
            self.device = torch_directml.device(dml_device_id)
            self.device_name = f"AMD/Intel DirectML ({torch_directml.device_name(dml_device_id)})"
        else:
            self.device = torch.device("cpu")
            self.device_name = "CPU (Процессор)"

        self.setup_styles()
        self.build_ui()
        self.load_model()

    def setup_styles(self):
        style = ttk.Style()
        style.theme_use('clam')
        style.configure("TProgressbar", thickness=20, background=SUCCESS, troughcolor="#E0E0E0")

    def build_ui(self):
        sidebar = tk.Frame(self.root, bg="#2C3E50", width=250)
        sidebar.pack(side=tk.LEFT, fill=tk.Y)
        sidebar.pack_propagate(False)

        lbl_logo = tk.Label(sidebar, text="ТОПФ AI", font=("Segoe UI Black", 24), bg="#2C3E50", fg="FFFFFF")
        lbl_logo.pack(pady=(30, 5))
        lbl_sub = tk.Label(sidebar, text="Анализ влажности", font=("Segoe UI", 10), bg="#2C3E50",
fg=TEXT_MUTED)
        lbl_sub.pack(pady=(0, 40))

        self.create_nav_button(sidebar, "1. Обучить модель", PRIMARY, self.show_train_view)
        self.create_nav_button(sidebar, "2. Определить влажность", SUCCESS, self.show_predict_view)
        self.create_nav_button(sidebar, "3. Протестировать выборку", WARNING, self.show_eval_view)

        self.model_status_lbl = tk.Label(sidebar, text="Загрузка модели...", font=("Segoe UI", 9), bg="#2C3E50",
fg=TEXT_MUTED)

```

```

self.model_status_lbl.pack(side=tk.BOTTOM, pady=10)

lbl_dev = tk.Label(sidebar, text=f"Устройство: \n{self.device_name}", font=("Segoe UI", 8), bg="#2C3E50",
fg=TEXT_MUTED)
lbl_dev.pack(side=tk.BOTTOM, pady=(0, 10))

self.main_content = tk.Frame(self.root, bg=BG_COLOR)
self.main_content.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

self.header_lbl = tk.Label(self.main_content, text="Добро пожаловать", font=("Segoe UI", 20, "bold"),
bg=BG_COLOR, fg=TEXT_MAIN)
self.header_lbl.pack(anchor="w", padx=30, pady=(30, 20))

self.view_container = tk.Frame(self.main_content, bg=BG_COLOR)
self.view_container.pack(fill=tk.BOTH, expand=True, padx=30, pady=(0, 30))

self.current_widgets = []
self.show_predict_view()

def create_nav_button(self, parent, text, color, command):
    btn = tk.Button(parent, text=text, font=("Segoe UI", 12, "bold"), bg=color, fg="FFFFFF",
activebackground="FFFFFF", activeforeground=color,
relief=tk.FLAT, borderwidth=0, pady=12, cursor="hand2", command=command)
    btn.pack(fill=tk.X, padx=15, pady=10)
    return btn

def clear_view(self):
    for w in self.current_widgets: w.destroy()
    self.current_widgets.clear()

def create_card(self, parent, title, value, color):
    card = tk.Frame(parent, bg=PANEL_BG, highlightbackground="#E2E8F0", highlightthickness=1, bd=0)
    lbl_title = tk.Label(card, text=title, font=("Segoe UI", 10, "bold"), bg=PANEL_BG, fg=TEXT_MUTED)
    lbl_title.pack(anchor="w", padx=15, pady=(15, 0))

    lbl_val = tk.Label(card, text=value, font=("Segoe UI", 24, "bold"), bg=PANEL_BG, fg=color)
    lbl_val.pack(anchor="w", padx=15, pady=(0, 15))
    return card, lbl_val

def load_model(self):
    if os.path.exists(MODEL_PATH):
        try:
            self.model = models.resnet34(weights=None)
            self.model.fc = nn.Linear(self.model.fc.in_features, 1)
            self.model.load_state_dict(torch.load(MODEL_PATH, map_location=self.device, weights_only=False))
            self.model.to(self.device)
            self.model.eval()
            self.model_status_lbl.config(text="✓ Нейросеть готова к работе", fg=SUCCESS)
        except Exception as e:
            self.model_status_lbl.config(text="Ошибка загрузки модели", fg="#E74C3C")
    else:
        self.model_status_lbl.config(text="⚠ Модель не найдена", fg=WARNING)

def show_train_view(self):
    self.clear_view()
    self.header_lbl.config(text="Обучение модели")

    card = tk.Frame(self.view_container, bg=PANEL_BG, highlightbackground="#E2E8F0", highlightthickness=1)
    card.pack(fill=tk.X, pady=10)
    self.current_widgets.append(card)

    info = tk.Label(card,

```

```

        text="Запустить процесс переобучения нейросети на ваших данных",
        font=("Segoe UI", 12), bg=PANEL_BG, fg=TEXT_MAIN, justify=tk.LEFT)
info.pack(padx=20, pady=20)

self.train_progress = ttk.Progressbar(card, orient="horizontal", mode="determinate", style="TProgressbar")
self.lbl_train_status = tk.Label(card, text="Готов к запуску", font=("Segoe UI", 12, "bold"), bg=PANEL_BG,
                                fg=PRIMARY)
self.lbl_train_status.pack(pady=(0, 10))

btn_start = tk.Button(card, text="▶ ЗАПУСТИТЬ ОБУЧЕНИЕ", font=("Segoe UI", 12, "bold"), bg=PRIMARY,
fg="#FFF",
                        relief=tk.FLAT, padx=20, pady=10, cursor="hand2", command=self.mock_training_process)
btn_start.pack(pady=(0, 20))

def mock_training_process(self):
    self.train_progress.pack(fill=tk.X, padx=40, pady=20)
    self.train_progress["value"] = 0
    self.lbl_train_status.config(text="Подготовка данных...", fg=WARNING)

    threading.Thread(target=self._real_training_worker, daemon=True).start()

def _real_training_worker(self):
    ROOT_DIR = "dataset_train"
    EPOCHS = 20
    BATCH_SIZE = 8
    LEARNING_RATE = 1e-4

    try:
        train_loader, val_loader = create_dataloaders(ROOT_DIR, batch_size=BATCH_SIZE, val_split=0.2)
    except Exception as e:
        self.root.after(0, lambda: self.lbl_train_status.config(text=f"Ошибка данных: {e}", fg="#E74C3C"))
        return

    self.root.after(0, lambda: self.lbl_train_status.config(text="Инициализация модели..."))

    train_model = models.resnet34(weights=models.ResNet34_Weights.IMAGENET1K_V1)
    train_model.fc = nn.Linear(train_model.fc.in_features, 1)
    nn.init.constant_(train_model.fc.bias, 0.5)
    train_model = train_model.to(self.device)

    criterion = nn.L1Loss()
    optimizer = optim.AdamW(train_model.parameters(), lr=LEARNING_RATE, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3)

    best_val_mae = float('inf')
    history_train_loss = []
    history_val_loss = []

    for epoch in range(1, EPOCHS + 1):
        train_model.train()
        train_loss = 0.0
        total_batches = len(train_loader)

        for i, (images, targets) in enumerate(train_loader):
            images, targets = images.to(self.device), targets.to(self.device)

            optimizer.zero_grad()
            outputs = train_model(images)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()

```

```

    real_loss = loss.item() * (TARGET_MAX - TARGET_MIN)
    train_loss += real_loss * images.size(0)

    progress = ((epoch - 1) + (i / total_batches)) / EPOCHS * 100
    status_text = f"Обучение: Эпоха {epoch}/{EPOCHS} | Train MAE: {real_loss:.1f}"

    self.root.after(0, lambda p=progress, t=status_text: self.update_train_ui(p, t))

    avg_train_loss = train_loss / len(train_loader.dataset)

    self.root.after(0, lambda e=epoch: self.update_train_ui(((e - 0.5) / EPOCHS * 100),
        f"Валидация: Эпоха {e}/{EPOCHS}..."))

    train_model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, targets in val_loader:
            images, targets = images.to(self.device), targets.to(self.device)
            outputs = train_model(images)
            loss = criterion(outputs, targets)
            real_loss = loss.item() * (TARGET_MAX - TARGET_MIN)
            val_loss += real_loss * images.size(0)

    avg_val_loss = val_loss / len(val_loader.dataset)
    history_train_loss.append(avg_train_loss)
    history_val_loss.append(avg_val_loss)

    scheduler.step(avg_val_loss / (TARGET_MAX - TARGET_MIN))

    if avg_val_loss < best_val_mae:
        best_val_mae = avg_val_loss
        torch.save(train_model.state_dict(), MODEL_PATH)

    self.root.after(0, lambda: self.finish_training_ui(history_train_loss, history_val_loss))

def update_train_ui(self, progress_val, text):
    self.train_progress["value"] = progress_val
    self.lbl_train_status.config(text=text)

def finish_training_ui(self, t_loss, v_loss):
    self.train_progress["value"] = 100
    self.lbl_train_status.config(text="✓ Обучение успешно завершено! Модель сохранена.", fg=SUCCESS)

    self.load_model()

    plt.figure(figsize=(10, 6))
    plt.plot(range(1, len(t_loss) + 1), t_loss, label='Train MAE', marker='o')
    plt.plot(range(1, len(v_loss) + 1), v_loss, label='Val MAE', marker='x')
    plt.title('График обучения модели (MAE)')
    plt.xlabel('Эпоха')
    plt.ylabel('MAE')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig('training_history.png', dpi=300)
    plt.show(block=False)

def show_predict_view(self):
    self.clear_view()
    self.header_lbl.config(text="Анализ фотографии")

    row_frame = tk.Frame(self.view_container, bg=BG_COLOR)

```

```

row_frame.pack(fill=tk.BOTH, expand=True)
self.current_widgets.append(row_frame)

left_panel = tk.Frame(row_frame, bg=PANEL_BG, highlightbackground="#E2E8F0", highlightthickness=1)
left_panel.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 15))

self.img_lbl = tk.Label(left_panel, bg="#E8ECF1", text="Нажмите 'Загрузить фото'", font=("Segoe UI", 12),
                        fg=TEXT_MUTED)
self.img_lbl.pack(fill=tk.BOTH, expand=True, padx=15, pady=15)

btn_load = tk.Button(left_panel, text="📁 Выбрать фото морфа", font=("Segoe UI", 12, "bold"), bg=SUC-
CESS,
                    fg="#FFF",
                    relief=tk.FLAT, pady=10, cursor="hand2", command=self.process_single_image)
btn_load.pack(fill=tk.X, padx=15, pady=(0, 15))

right_panel = tk.Frame(row_frame, bg=BG_COLOR)
right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

c1, self.lbl_pred_res = self.create_card(right_panel, "ПРЕДСКАЗАННАЯ ВЛАЖНОСТЬ", "---", SUCCESS)
c1.pack(fill=tk.X, pady=(0, 15))

c2, self.lbl_true_res = self.create_card(right_panel, "ФАКТИЧЕСКАЯ ВЛАЖНОСТЬ", "---", TEXT_MUTED)
c2.pack(fill=tk.X)

self.lbl_pred_status = tk.Label(right_panel, text="Ожидание фото...", font=("Segoe UI", 11), bg=BG_COLOR,
                                fg=TEXT_MUTED)
self.lbl_pred_status.pack(pady=20)

def process_single_image(self):
    if self.model is None:
        messagebox.showerror("Ошибка", "Модель не загружена!")
        return

    file_path = filedialog.askopenfilename(filetypes=[("Images", "*.jpg *.png")])
    if not file_path: return

    img = Image.open(file_path)
    img.thumbnail((350, 350))
    img_tk = ImageTk.PhotoImage(img)
    self.img_lbl.config(image=img_tk, text="")
    self.img_lbl.image = img_tk
    self.lbl_pred_status.config(text="Анализ нейросетью...", fg=PRIMARY)
    self.root.update()

    try:
        img_tensor = Image.open(file_path).convert('RGB')
        tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize(mean=MEAN, std=STD)])
        crops = [tf(transforms.RandomCrop(512)(img_tensor)) for _ in range(5)]
        batch = torch.stack(crops).to(self.device)

        with torch.no_grad():
            preds = self.model(batch)

        real_val = preds.mean().item() * (TARGET_MAX - TARGET_MIN) + TARGET_MIN
        self.lbl_pred_res.config(text=f"{real_val:.1f}")

        folder_name = os.path.basename(os.path.dirname(file_path))
        true_val = TARGET_MAPPING.get(folder_name)

        if true_val is not None:
            self.lbl_true_res.config(text=f"{true_val:.1f}", fg=INFO)

```

```

        error = abs(true_val - real_val)
        self.lbl_pred_status.config(text=f"Анализ завершен. Погрешность: {error:.1f}", fg=TEXT_MAIN)
    else:
        self.lbl_true_res.config(text="Неизвестно", fg=TEXT_MUTED)
        self.lbl_pred_status.config(text="Анализ завершен. Класс фото неизвестен.", fg=TEXT_MAIN)

except Exception as e:
    self.lbl_pred_status.config(text=f"Ошибка: {e}", fg="#E74C3C")

def show_eval_view(self):
    self.clear_view()
    self.header_lbl.config(text="Оценка качества на тестовой выборке")

    ctrl_card = tk.Frame(self.view_container, bg=PANEL_BG, highlightbackground="#E2E8F0", highlightthickness=1)
    ctrl_card.pack(fill=tk.X, pady=(0, 20))
    self.current_widgets.append(ctrl_card)

    self.eval_prog = ttk.Progressbar(ctrl_card, orient="horizontal", mode="determinate", style="TProgressbar")
    self.eval_status = tk.Label(ctrl_card, text="Готов к проверке", font=("Segoe UI", 12), bg=PANEL_BG, fg=TEXT_MUTED)
    self.eval_status.pack(pady=(15, 5))

    btn_eval = tk.Button(ctrl_card, text="🌀 ЗАПУСТИТЬ ТЕСТИРОВАНИЕ", font=("Segoe UI", 12, "bold"), bg=WARNING, fg="#FFF", relief=tk.FLAT, padx=20, pady=10, cursor="hand2", command=self.run_evaluation)
    btn_eval.pack(pady=(5, 15))

    grid_frame = tk.Frame(self.view_container, bg=BG_COLOR)
    grid_frame.pack(fill=tk.BOTH, expand=True)
    self.current_widgets.append(grid_frame)
    grid_frame.columnconfigure(0, weight=1)
    grid_frame.columnconfigure(1, weight=1)

    c1, self.lbl_mae = self.create_card(grid_frame, "MAE (Абс. ошибка)", "---", PRIMARY)
    c2, self.lbl_rmse = self.create_card(grid_frame, "RMSE (Квадр. ошибка)", "---", "#E74C3C")
    c3, self.lbl_mape = self.create_card(grid_frame, "MAPE (В процентах)", "---", WARNING)
    c4, self.lbl_r2 = self.create_card(grid_frame, "R² (Детерминация)", "---", SUCCESS)

    c1.grid(row=0, column=0, sticky="nsew", padx=10, pady=10)
    c2.grid(row=0, column=1, sticky="nsew", padx=10, pady=10)
    c3.grid(row=1, column=0, sticky="nsew", padx=10, pady=10)
    c4.grid(row=1, column=1, sticky="nsew", padx=10, pady=10)

def run_evaluation(self):
    if self.model is None:
        messagebox.showerror("Ошибка", "Модель не загружена!")
        return

    if not os.path.exists(TEST_DIR):
        messagebox.showerror("Ошибка", f"Папка {TEST_DIR} не найдена!")
        return

    self.eval_prog.pack(fill=tk.X, padx=40, pady=(0, 10))
    self.eval_status.config(text="Чтение файлов...", fg=PRIMARY)

    for lbl in [self.lbl_mae, self.lbl_rmse, self.lbl_mape, self.lbl_r2]:
        lbl.config(text="...")

    threading.Thread(target=self._eval_worker, daemon=True).start()

```



```

def _eval_worker(self):
    try:
        true_vals, pred_vals = [], []
        tf = transforms.Compose(
            [transforms.CenterCrop(512), transforms.ToTensor(), transforms.Normalize(mean=MEAN, std=STD)])

        all_files = []
        for k, val in TARGET_MAPPING.items():
            p = os.path.join(TEST_DIR, k)
            if os.path.exists(p):
                for f in os.listdir(p):
                    if f.lower().endswith(('.jpg', '.png', '.jpeg')):
                        all_files.append((os.path.join(p, f), val))

        total = len(all_files)
        if total == 0:
            self.root.after(0, lambda: self.eval_status.config(text="Тестовая папка пуста!", fg="#E74C3C"))
            return

        for i, (path, t_val) in enumerate(all_files):
            img = Image.open(path).convert('RGB')
            tensor = tf(img).unsqueeze(0).to(self.device)
            with torch.no_grad():
                p_val = self.model(tensor).item() * (TARGET_MAX - TARGET_MIN) + TARGET_MIN
            true_vals.append(t_val)
            pred_vals.append(p_val)

        self.root.after(0, lambda v=(i + 1) / total * 100, c=i + 1: self.update_eval_prog(v, c, total))

        t_arr, p_arr = np.array(true_vals), np.array(pred_vals)
        mae = mean_absolute_error(t_arr, p_arr)
        rmse = np.sqrt(mean_squared_error(t_arr, p_arr))
        mape = mean_absolute_percentage_error(t_arr, p_arr) * 100
        r2 = r2_score(t_arr, p_arr)

        self.root.after(0, lambda: self.finish_eval(mae, rmse, mape, r2, t_arr, p_arr))

    except Exception as e:
        self.root.after(0, lambda: self.eval_status.config(text=f"Ошибка: {e}", fg="#E74C3C"))

def update_eval_prog(self, prog_val, current, total):
    self.eval_prog["value"] = prog_val
    self.eval_status.config(text=f"Обработано {current} из {total} фото...")

def finish_eval(self, mae, rmse, mape, r2, true_arr, pred_arr):
    self.eval_status.config(text="Формирование графических отчетов...", fg=WARNING)
    self.root.update()

    self.lbl_mae.config(text=f"{mae:.1f}")
    self.lbl_rmse.config(text=f"{rmse:.1f}")
    self.lbl_mape.config(text=f"{mape:.1f} %")
    self.lbl_r2.config(text=f"{r2:.3f}")

    self.show_evaluation_plots(true_arr, pred_arr)

def show_evaluation_plots(self, true_arr, pred_arr):
    fig1, axs = plt.subplots(1, 2, figsize=(16, 6))

    axs[0].scatter(true_arr, pred_arr, alpha=0.6, color='blue', edgecolors='k')
    min_v, max_v = min(true_arr), max(true_arr)
    axs[0].plot([min_v, max_v], [min_v, max_v], 'r--', lw=2, label='Идеальное совпадение')
    axs[0].set_title('Истинная vs Предсказанная влажность', fontsize=14)

```

```

axs[0].set_xlabel('Истинная влажность торфа', fontsize=12)
axs[0].set_ylabel('Предсказание модели', fontsize=12)
axs[0].grid(True, linestyle='--', alpha=0.7)
axs[0].legend()

errors = pred_arr - true_arr
axs[1].hist(errors, bins=15, color='orange', edgecolor='black', alpha=0.7)
axs[1].axvline(0, color='red', linestyle='dashed', linewidth=2, label='Нулевая ошибка')
axs[1].set_title('Распределение ошибок (Предсказание - Истина)', fontsize=14)
axs[1].set_xlabel('Смещение ошибки', fontsize=12)
axs[1].set_ylabel('Количество фото', fontsize=12)
axs[1].grid(True, linestyle='--', alpha=0.7)
axs[1].legend()

fig1.tight_layout()
fig1.savefig('model_evaluation_report.png', dpi=300)

class_keys = sorted(list(TARGET_MAPPING.keys()))
fig2, axes = plt.subplots(2, len(class_keys), figsize=(26, 8))
fig2.suptitle('Проверка предсказаний модели (7 классов влажности)', fontsize=18, fontweight='bold')

tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize(mean=MEAN, std=STD)])

for col_idx, folder_name in enumerate(class_keys):
    true_val = TARGET_MAPPING[folder_name]
    f_path = os.path.join(TEST_DIR, folder_name)

    files = [f for f in os.listdir(f_path) if f.lower().endswith(('jpg', 'jpeg', 'png')) if os.path.isfile(f_path)]

    if len(files) >= 2:
        selected_files = random.sample(files, 2)
    elif len(files) == 1:
        selected_files = [files[0], files[0]]
    else:
        selected_files = [None, None]

    for row_idx in range(2):
        ax = axes[row_idx, col_idx]
        file_name = selected_files[row_idx]

        if file_name is None:
            ax.axis('off')
            ax.set_title(f'Нет фото\n({true_val})', fontsize=10)
            continue

        img_path = os.path.join(f_path, file_name)

        try:
            img = Image.open(img_path).convert('RGB')
            crops = [tf(transforms.RandomCrop(512)(img)) for _ in range(5)]
            batch = torch.stack(crops).to(self.device)

            with torch.no_grad():
                preds = self.model(batch)

            pred_val = preds.mean().item() * (TARGET_MAX - TARGET_MIN) + TARGET_MIN
            error = abs(true_val - pred_val)

            img_show = img.copy()
            img_show.thumbnail((800, 800))
            ax.imshow(img_show)

```

```

ax.axis('off')

color = 'green' if error < 100 else 'red'
ax.set_title(f"Истина: {true_val}\nПредсказ.: {pred_val:.1f}\nОшибка: {error:.1f}", fontsize=12,
            color=color, fontweight='bold')
except Exception:
    ax.axis('off')

fig2.tight_layout(rect=[0, 0.03, 1, 0.95])
fig2.savefig('test_predictions_grid.png', dpi=300, bbox_inches='tight')

self.eval_status.config(text="✓ Тестирование завершено! Графики открыты.", fg=SUCCESS)

plt.show(block=False)

if __name__ == "__main__":
    root = tk.Tk()
    app = DashboardApp(root)
    root.mainloop()

```

ПРИЛОЖЕНИЕ Б

(обязательное)

Руководство пользователя

Программный комплекс «ТОРФ *AI*» предназначен для автоматизированного визуального анализа цифровых фотографий торфа с целью бесконтактного определения уровня его влажности (в диапазоне от 1720 до 2650 условных единиц). Приложение предоставляет пользователю удобный графический интерфейс для обучения нейросетевой модели, оценки одиночных изображений и проведения комплексного тестирования на наборах данных.

Для корректного функционирования программы необходимо соблюдение следующих условий:

- аппаратное обеспечение: процессор (*CPU*) с частотой от 2,0 ГГц, при этом рекомендуется наличие графического ускорителя (*GPU*) от *NVIDIA* (с поддержкой *CUDA*) или *AMD/Intel* (с поддержкой *DirectML*) для существенного ускорения нейросетевых вычислений;
- программное обеспечение: операционная система *Windows* 10/11 или *Linux* с установленным интерпретатором *Python* версии 3,8 или выше;
- структура директорий: в корневой папке программы (совместно с исполняемым файлом *app.py*) должны располагаться папки *dataset_train* и *dataset_test*, содержащие внутри себя директории с названиями классов-значений,

Запуск программы осуществляется путем выполнения скрипта *app.py* через командную строку или среду разработки (*IDE*). Главное окно программы логически разделено на две области:

- левая навигационная панель: содержит логотип, главные кнопки переключения режимов работы, индикатор статуса загрузки модели и информацию об используемом вычислительном устройстве;
- правая рабочая область: динамически отображает элементы управления и результаты работы для выбранного режима.

Режим «обучить модель» предназначен для инициализации процесса тренировки нейронной сети на наборе данных *dataset_train*. После выбора данного режима на навигационной панели пользователю необходимо запустить алгоритм соответствующей кнопкой в рабочей области. Вычислительный процесс выполняется в фоновом режиме, при этом в окне отображается индикатор выполнения и текстовое информирование о текущей эпохе и значении ошибки (*MAE*). По завершении тренировки веса модели автоматически сохраняются, статус нейросети обновляется, а на экран выводится график истории обучения.

Режим «определить влажность» используется для анализа произвольных фотографий торфа. После перехода в данный раздел пользователю предлагается выбрать изображение формата *.jpg*, *.jpeg* или *.png* через стандартное системное

окно. Программа автоматически обрабатывает снимок, выводя его миниатюру в левой части экрана. В правой части формируются следующие информационные карточки:

- «предсказанная влажность» (выделяется зеленым цветом) отражает итоговый результат математического расчета нейросети;
- «фактическая влажность» выводится для сравнения в том случае, если фотография была загружена из папки с заранее известным классом (в противном случае отображается статус «неизвестно»);
- итоговая текстовая сводка, расположенная внизу, фиксирует вычисленную абсолютную погрешность предсказания.

Режим «протестировать выборку» позволяет оценить качество работы обученной модели на всем массиве данных, находящемся в папке *dataset_test*. Инициализация проверки осуществляется кнопкой запуска тестирования. По мере заполнения индикатора загрузки программа скрыто анализирует все снимки в директории, после чего выводит на экран четыре ключевые метрики качества:

- *MAE* (абсолютная ошибка в условных единицах влажности);
- *RMSE* (среднеквадратичная ошибка);
- *MAPE* (относительная ошибка в процентах);
- R^2 (коэффициент детерминации).

Дополнительно по итогам комплексного тестирования автоматически генерируются и открываются два аналитических окна: график распределения ошибок (включающий гистограмму и диаграмму рассеяния) и визуальная сетка с примерами предсказаний для каждого класса влажности. Данные графики также автоматически сохраняются в корневую папку проекта в формате *.png*.

ПРИЛОЖЕНИЕ В

(обязательное)

Руководство программиста

Программное обеспечение разработано на языке программирования *Python* с использованием парадигмы объектно-ориентированного программирования. Вычислительное ядро системы, отвечающее за работу нейронной сети, построено на базе фреймворка *PyTorch*. Визуальная оболочка реализована с использованием стандартной библиотеки *Tkinter*.

В программе реализована кросс-аппаратная поддержка ускорения вычислений: помимо стандартных процессоров (*CPU*) и графических ускорителей *NVIDIA (CUDA)*, поддерживается *API DirectML* (через модуль *torch_directml*) для выполнения тензорных операций на видеокартах *AMD* и *Intel*.

Исходный код программы содержится в едином файле *app.py* и логически разделен на следующие функциональные блоки:

- блок импорта зависимостей и инициализации глобальных констант;
- блок подготовки и фильтрации данных (функция *is_good_image*, класс *PeatDataset*, функция *create_dataloaders*);
- блок графического интерфейса и управления логикой (класс *DashboardApp*);
- фоновые обработчики математических вычислений (методы *_real_training_worker* и *_eval_worker*).

Глобальный словарь *TARGET_MAPPING* выполняет функцию сопоставления строковых названий папок с их физическими значениями влажности (от 1720,0 до 2650,0).

Для обеспечения стабильности градиентного спуска при обучении нейросети, целевые значения влажности нормализуются в диапазон $[0, 1]$ по формуле *Min-Max Scaling* (с использованием констант *TARGET_MIN* и *TARGET_MAX*). При выводе результатов пользователю применяется обратное алгебраическое преобразование.

Функция *is_good_image* использует библиотеку *Pillow* и модуль *ImageStat* для отбраковки неинформативных кадров. Изображение бракуется, если средняя яркость пикселей выходит за пределы $[30, 230]$ или дисперсия контрастности ниже 15.

Класс *PeatDataset*, унаследованный от *torch.utils.data.Dataset*, отвечает за чтение изображений с диска. Функция *create_dataloaders* реализует разделение датасета на обучающую и валидационную выборки (по умолчанию 80/20) и применяет аугментации: *RandomCrop* (512×512), *RandomHorizontalFlip*, *ColorJitter* и нормализацию по *ImageNet*.

В качестве базовой архитектуры применяется модель *ResNet34* из модуля *torchvision.models*. Для адаптации архитектуры под задачу регрессии, терминальный слой классификатора (*model.fc*) заменяется на линейный слой с одним выходным нейроном: *nn.Linear(in_features, 1)*.

Важной алгоритмической особенностью является инициализация вектора смещения (*bias*) выходного слоя константой 0.5 (середина нормализованного диапазона), что существенно ускоряет первоначальную сходимость модели.

Для оптимизации весовых коэффициентов используется алгоритм *AdamW* с коэффициентом обучения $1e-4$ и функцией потерь *L1Loss* (средняя абсолютная ошибка). Регулировка шага обучения осуществляется планировщиком *ReduceLROnPlateau*.

Для предотвращения блокировки главного цикла обработки событий *Tkinter* (*mainloop*) во время выполнения ресурсоемких тензорных операций, процессы обучения и тестирования запускаются в изолированных фоновых потоках с помощью модуля *threading*. Взаимодействие фоновых потоков с элементами графического интерфейса реализуется исключительно потокобезопасным методом *self.root.after()*.

Инференс (предсказание на одиночном изображении) реализован в методе *process_single_image*. Применяется стратегия ансамблирования: из исходного изображения стохастически вырезаются 5 фрагментов (тензоров), каждый прогоняется через модель, после чего полученные предсказания усредняются (*preds.mean()*). Это повышает робастность системы к оптическим помехам.

Модуль тестирования (метод *_eval_worker*) использует библиотеку *scikit-learn* (*sklearn.metrics*) для вычисления метрик *MAE*, *RMSE*, *MAPE* и R^2 . Построение аналитических графиков (диаграмма рассеяния, гистограмма распределения ошибок, сетка визуального контроля предсказаний) выполняется с помощью библиотеки *Matplotlib*. Сгенерированные графики сохраняются в корневую директорию проекта.

ПРИЛОЖЕНИЕ Г

(обязательное)

Внешний вид графического интерфейса программы

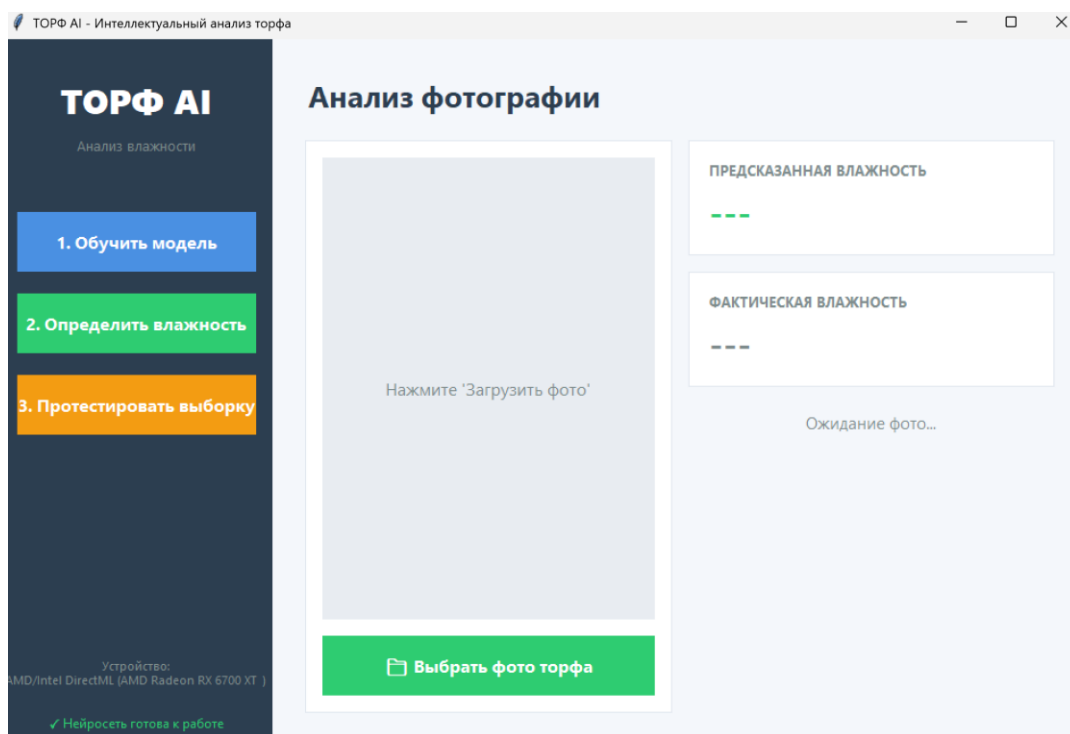


Рисунок Г.1 – Главное окно приложения

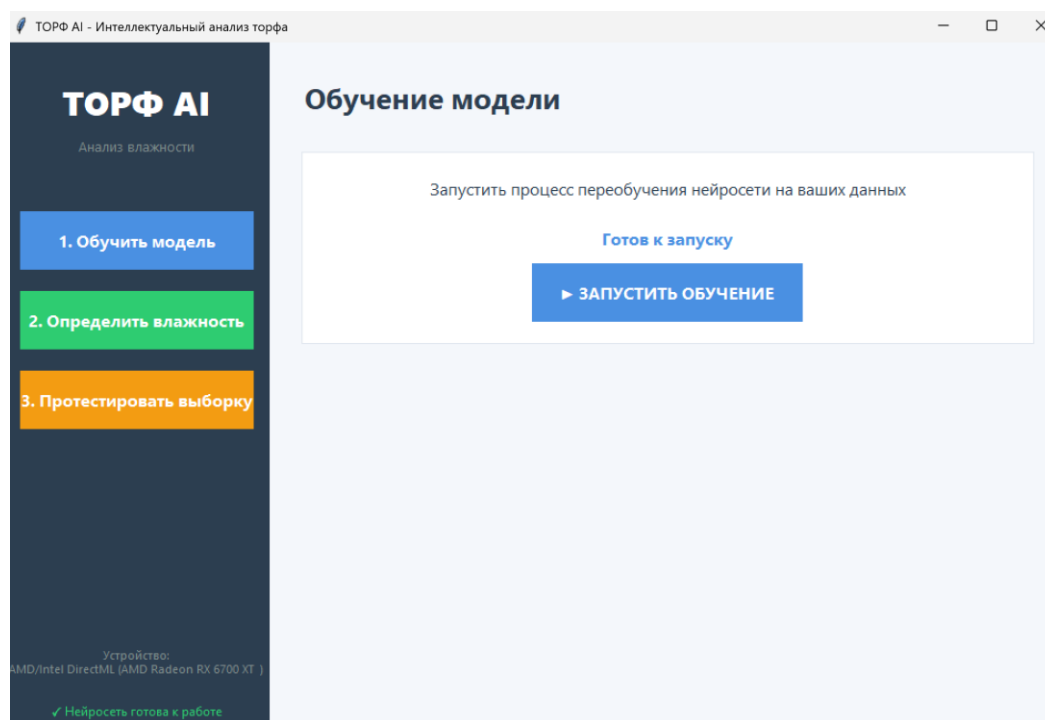


Рисунок Г.2 – Окно обучения модели

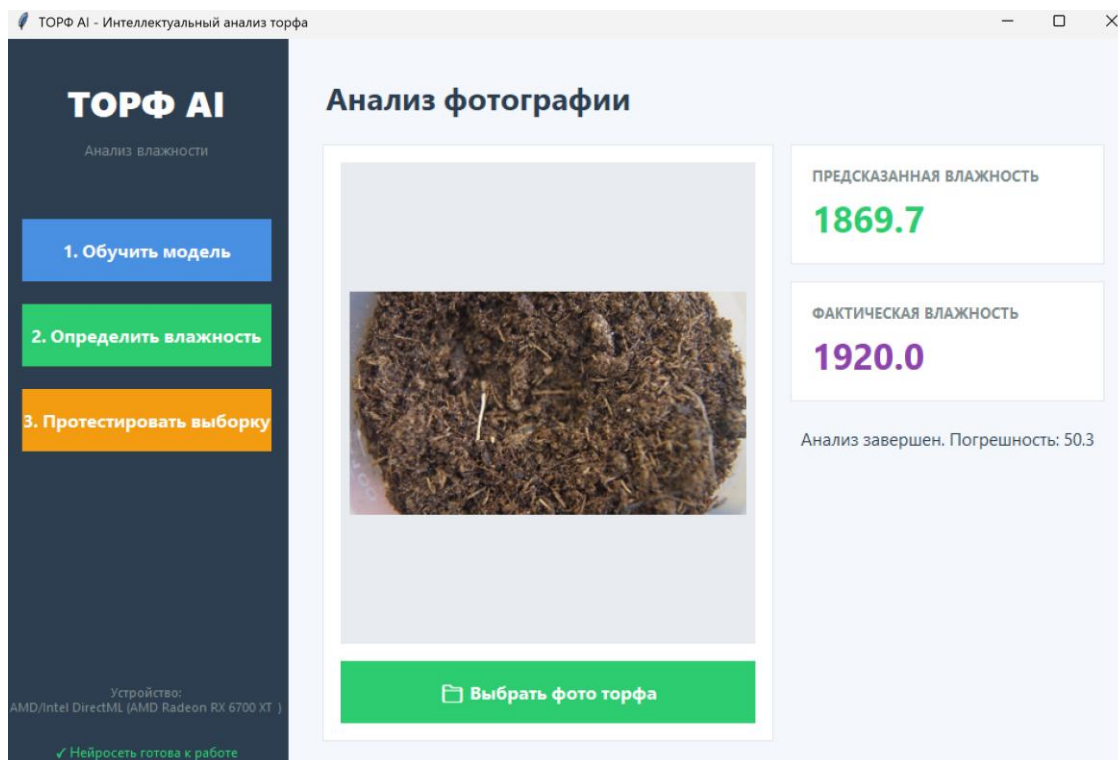


Рисунок Г.3 – Определение влажности торфа по фотографии

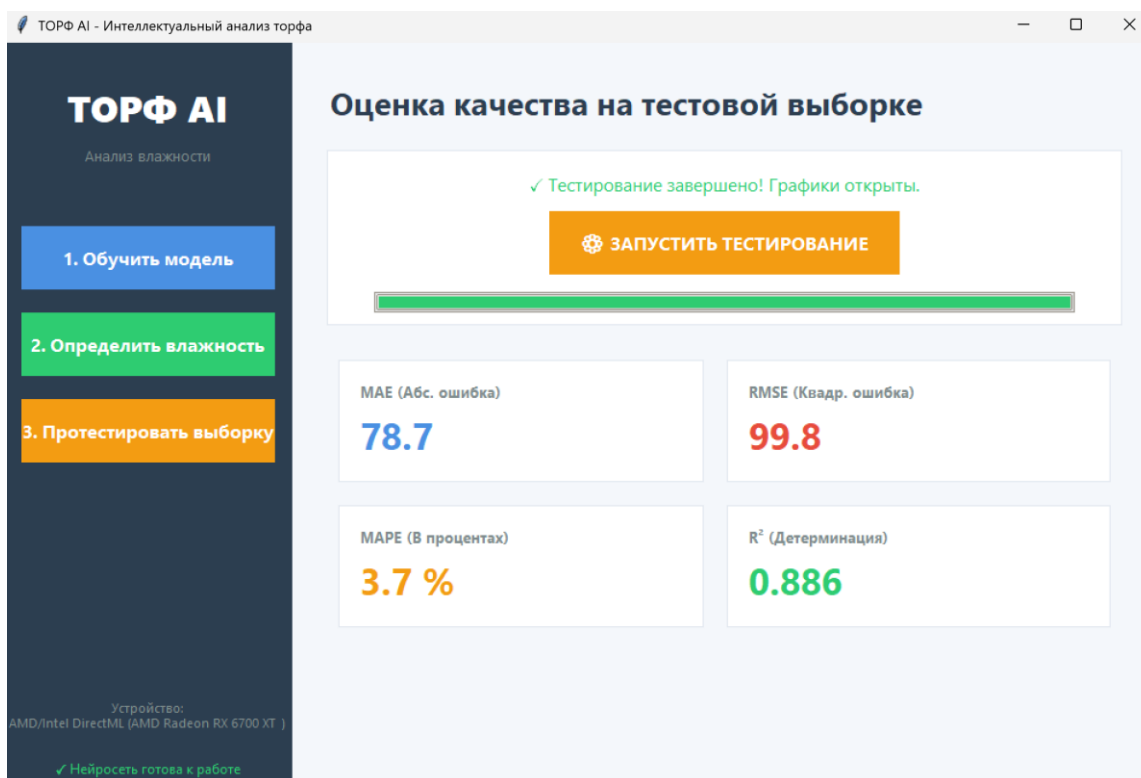


Рисунок Г.4 – Тестирование модели